

Computer Vision

RDFIA Section1

Zhang Zhile & Moubarak Carine



Sorbonne Université
M2 DAC
October 29, 2024

Contents

1	Introduction to Neural Networks	2
1.1	Theoretical foundation	2
1.1.1	Supervised dataset	2
1.1.2	Network architecture (forward)	2
1.1.3	Loss function	4
1.1.4	Optimization algorithm	4
1.2	Implementation	11
1.2.1	Forward and backward manual	11
1.2.2	Simplification of the backward pass with <code>torch.autograd</code>	13
1.2.3	Simplification of the forward pass with <code>torch.nn</code> layers	15
1.2.4	Simplification of SGD with <code>torch.optim</code>	17
1.2.5	MNIST Application	18
1.2.6	Bonus: SVM	20
2	Convolutional Neural Networks	22
2.1	Introduction to Convolutional Neural Networks	22
2.1.1	Convolutional layers	22
2.2	Training from scratch of the model	24
2.2.1	Network architecture	24
2.2.2	Network learning	26
2.3	Results improvements	28
2.3.1	Standardization of examples	28
2.3.2	Increase in training examples by data augmentation	29
2.3.3	Variants on the optimization algorithm	32
2.3.4	Regularization of the network by dropout	33
2.3.5	Use of batch normalization	35
3	Transformers	37
3.1	Self-Attention	37
3.2	Multi-head self-attention	38
3.3	Transformer block	39
3.4	Full ViT model	40
3.5	Experiment on MNIST	41
3.5.1	Quick function	41
3.5.2	Experimenting <code>embed_dim</code>	41
3.5.3	Experimenting <code>patch_size</code>	42
3.5.4	Experimenting <code>nb_blocks</code>	42
3.6	Larger transformers	43

Chapter 1

Introduction to Neural Networks

1.1 Theoretical foundation

1.1.1 Supervised dataset

- ***Q1:** What are the train, val, and test sets used for?

The train set is used to train the model, to learn patterns from the data. The validation set helps to check the performance and to also adjust the hyperparameters without touching the test set. And the test set helps evaluating the model, to see how the model performs on real world tasks.

- **Q2:** What is the influence of the number of examples N ?

The larger the N , the better the model can learn and generalize.

1.1.2 Network architecture (forward)

- **Q3:** Why is it important to add activation functions between linear transformations?

Activation functions are added between linear transformations to introduce non-linearity to allow the model to learn complex patterns. Without them, the network would only be able to model simple, linear relations, which limits its effectiveness.

- ***Q4:** What are the sizes n_x , n_h , n_y in Figure 1? In practice, how are these sizes chosen?

- n_x : The size of the **input layer**, which is the number of features in the input data. Each sample has a feature vector $x(i)$ of size n_x , so this represents the number of input neurons. So it is determined by the dataset.

- n_h : The size of the **hidden layer**, which refers to the number of neurons in the hidden layer(s). There can be multiple hidden layers, each with its own number of neurons. n_h controls the capacity of the network to learn complex patterns.

This one is chosen experimentally and based on the complexity of the problem. More neurons or layers allow the model to capture more complex relationships, but too many can lead to overfitting.

- n_y : The size of the **output layer**, corresponding to the number of target classes. In a classification task with one-hot encoding, n_y equals the number of possible classes.

- **Q5:** What do the vectors $\hat{y}$ and y represent? What is the difference between these two quantities?

$\hat{y}$ (y_hat) represents the **predicted output** of the model. y represents the **actual output** (ground truth). The difference between these two is that y is the true value or label, while $\hat{y}$ is the model's guess or prediction. The goal is to make $\hat{y}$ as close as possible to y .

- **Q6:** Why use a SoftMax function as the output activation function?

The SoftMax function is used in multi-class classification because it turns the raw scores from the model into probabilities that add up to 1. This helps us easily understand how likely each class is. SoftMax makes higher scores more important, helping the model make clearer decisions about which class to choose.

- **Question 7:** Write the mathematical equations allowing to perform the forward pass of the neural network, i.e. allowing to successively produce $\tilde{h}$, h , $\tilde{y}$, and $\hat{y}$ starting at x .

To perform a forward pass in a neural network, we use the following equations:

1. Hidden Layer Pre-Activation:

$$\tilde{h} = xW_h^T + b_h$$

2. Hidden Layer Activation (using activation function $\tanh$):

$$h = \tanh(\tilde{h})$$

3. Output Layer Pre-Activation:

$$\tilde{y} = hW_y^T + b_y$$

4. Output Layer Activation (using SoftMax):

$$\hat{y} = \text{SoftMax}(\tilde{y})$$

Here, $\tilde{h}$ is the input to the hidden layer, h is the activated output, $\tilde{y}$ is the output before applying SoftMax, and $\hat{y}$ is the final predicted probabilities.

1.1.3 Loss function

- **Question 8:** During training, how must the $\hat{y}_i$ vary to decrease the global loss function L for cross entropy and squared error?

During training, we want to minimize the loss function by making the predicted values, $\hat{y}_i$, closer to the actual target values, y_i .

For cross-entropy loss, $\hat{y}_i$ should move closer to the true target class. Specifically:

- When $y_i = 1$ (for the correct class), $\hat{y}_i$ should approach 1. The closer $\hat{y}_i$ is to 1, the lower the loss.
- When $y_i = 0$ (for the incorrect class), $\hat{y}_i$ should approach 0. The closer $\hat{y}_i$ is to 0, the lower the loss.

For squared error loss, $\hat{y}_i$ should simply get as close as possible to y_i . This means:

- If $\hat{y}_i$ is greater than y_i , it should decrease.
- If $\hat{y}_i$ is smaller than y_i , it should increase.

In both cases, the goal is to adjust $\hat{y}_i$ so that the overall loss is minimized, leading to a better-trained model.

1.1.4 Optimization algorithm

- **Question 10:** What seem to be the advantages and disadvantages of the various variants of gradient descent between the classic, mini-batch stochastic, and online stochastic versions? Which one seems the most reasonable to use in the general case?

Classic (Batch) Gradient Descent uses the entire dataset to compute the gradient before updating the model weights. One of its main advantages is stable convergence, as it provides accurate estimates of the gradient. However, it can be slow and computationally expensive, especially for large datasets, as it requires loading the entire dataset into memory for each update.

Stochastic Gradient Descent (SGD), updates the model weights using only one sample at a time. This leads to faster updates and can help the model escape local minima due to the inherent noise in the updates. However, the noisy nature of SGD makes it less stable and it may require careful tuning of the learning rate to achieve optimal results.

Mini-Batch Gradient Descent is a balanced method between the classic and stochastic methods by using small batches of data for each update. It is faster than classic gradient descent while being less noisy than SGD. However, it still requires tuning of both the batch size and learning rate, and if the batch size is too small, it may not fully exploit the dataset. In general, mini-batch gradient descent is often

considered the most reasonable choice.

- ***Question 11:** What is the influence of the learning rate η on learning?

The learning rate η determines the size of the steps taken towards the minimum of the loss function during optimization. If it is too small, the training process will be very slow, requiring many iterations to converge to a minimum. This can lead to long training times and may increase the risk of getting stuck in local minima without reaching the global minimum. On the other hand if η is too large, the updates can overshoot the minimum, causing the loss to oscillate or even diverge.

- ***Question 12:** Compare the complexity of calculating the gradients of the loss with respect to the parameters, using the naive approach and the backpropagation algorithm.

The naive approach involves computing the gradients for each parameter independently. For a neural network with L layers and N parameters, this method requires $O(N)$ forward passes and $O(N)$ backward passes. So the total computational complexity is $O(N^2)$, as each forward and backward pass involves iterating through all parameters.

In contrast, the backpropagation algorithm efficiently computes the gradients using the chain rule. Backpropagation calculates the gradients in a single backward pass after one forward pass. For a network with L layers, the complexity of computing the gradients using backpropagation is $O(N + L)$, as it involves iterating through the network layers and the parameters only once. That's why backpropagation is the preferred method for training deep neural networks, especially as the number of layers and parameters increases.

- **Question 13:** What criteria must the network architecture meet to allow such an optimization procedure?

For the backpropagation algorithm to work effectively, the network architecture must meet several criteria:

1. **Differentiability:** Each component of the network, including the activation functions, loss function, and layer functions, must be differentiable with respect to their parameters. This allows gradients to be calculated during training.
2. **Feedforward Structure:** The network should have a feedforward, layered architecture. This means that the output of each layer serves as the input to the next layer in a sequential manner. There should be no loops or recurrent connections, as these can complicate the gradient calculations.

These criteria ensure that the backpropagation algorithm can efficiently optimize the network during training.

- **Question 14:** Show that the loss can be simplified by the following formula:

$$\ell = - \sum_i y_i \tilde{y}_i + \log \left(\sum_i e^{\tilde{y}_i} \right)$$

The cross-entropy loss function is a widely used metric for evaluating the performance of classification models. It is defined as:

$$L(y, \hat{y}) = - \sum_{i=1}^{n_y} y_i \log(\hat{y}_i)$$

where y_i is the true label (one-hot encoded) and $\hat{y}_i$ is the predicted probability for class i .

To calculate $\hat{y}_i$, we use the SoftMax function, which converts a vector $\tilde{y}$ of raw scores (logits) into probabilities. The SoftMax function is defined as:

$$\hat{y}_i = \text{SoftMax}(\tilde{y}_i) = \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}}$$

Substituting this expression for $\hat{y}_i$ into the cross-entropy loss gives:

$$L(y, \hat{y}) = - \sum_{i=1}^{n_y} y_i \log \left(\frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}} \right)$$

This can be expanded as follows:

$$L(y, \hat{y}) = - \sum_{i=1}^{n_y} \left(y_i \log(e^{\tilde{y}_i}) - y_i \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right) \right)$$

Using the properties of logarithms, we can simplify this to:

$$= - \sum_{i=1}^{n_y} \left(y_i \tilde{y}_i - y_i \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right) \right)$$

Next, we observe that since y is one-hot encoded, the sum of all y_i is equal to 1:

$$\sum_{i=1}^{n_y} y_i = 1$$

Thus, we can simplify our loss function further by focusing on the key terms:

$$L(y, \hat{y}) = - \sum_{i=1}^{n_y} y_i \tilde{y}_i + \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right)$$

Finally, the expression for the cross-entropy loss can be summarized as:

$$L(y, \hat{y}) = - \sum_{i=1}^{n_y} y_i \tilde{y}_i + \log \left(\sum_{i=1}^{n_y} e^{\tilde{y}_i} \right)$$

- **Question 15:** Write the gradient of the loss (cross-entropy) relative to the intermediate output $\tilde{y}$.

The gradient of the cross-entropy loss l with respect to the intermediate output $\tilde{y}_i$ is calculated as follows:

Starting from the definition of the cross-entropy loss:

$$l(y, \hat{y}) = - \sum_{i=1}^{n_y} y_i \log(\hat{y}_i)$$

Substituting $\hat{y}_i = \text{SoftMax}(\tilde{y}_i)$, we have:

$$\frac{\partial l}{\partial \tilde{y}_i} = -y_i + \frac{\partial}{\partial \tilde{y}_i} \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j} \right) \frac{1}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}}$$

Using the quotient rule, we can differentiate the SoftMax function:

$$= -y_i + \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}}$$

Thus, we can express this as:

$$= -y_i + \text{SoftMax}(\tilde{y})_i$$

Combining these results, we find the gradient as a vector:

$$\nabla_{\tilde{y}} l = \begin{pmatrix} \frac{\partial l}{\partial \tilde{y}_1} \\ \vdots \\ \frac{\partial l}{\partial \tilde{y}_{n_y}} \end{pmatrix} = \begin{pmatrix} \text{SoftMax}(\tilde{y})_1 - y_1 \\ \vdots \\ \text{SoftMax}(\tilde{y})_{n_y} - y_{n_y} \end{pmatrix} = \hat{y} - y$$

where $\hat{y}$ is the vector of predicted probabilities from the SoftMax function and y is the vector of true labels.

- **Question 16:** Using the backpropagation, write the gradient of the loss with respect

to the weights of the output layer $\nabla W_y \ell$. Do the same for $\nabla b_y \ell$.

The gradient of the loss with respect to the weights of the output layer $\nabla_{W_y} \ell$ is given by:

$$\frac{\partial \ell}{\partial W_{y,i,j}} = \frac{\partial \ell}{\partial \tilde{y}_i} \frac{\partial \tilde{y}_i}{\partial W_{y,i,j}} = \frac{\partial \ell}{\partial \tilde{y}_i} \frac{\partial \sum_k (W_{y,i,k} h_k + b_{y,i})}{\partial W_{y,i,j}} = (\hat{y}_i - y_i) h_j$$

Thus, the gradient can be expressed in matrix form as:

$$\nabla_{W_y} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial \tilde{W}_{y,11}} & \cdots & \frac{\partial \ell}{\partial \tilde{W}_{y,1,n_h}} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial \tilde{W}_{y,n_y,1}} & \cdots & \frac{\partial \ell}{\partial \tilde{W}_{y,n_y,n_h}} \end{bmatrix} = \begin{bmatrix} \hat{y}_1 - y_1 \\ \vdots \\ \hat{y}_{n_y} - y_{n_y} \end{bmatrix}^T \begin{bmatrix} h_1 & \cdots & h_{n_h} & 0 & \cdots & 0 \\ 0 & h_1 & \cdots & h_{n_h} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \\ 0 & 0 & \cdots & h_1 & \cdots & h_{n_h} \end{bmatrix}$$

The gradient of the loss with respect to the bias of the output layer $\nabla_{b_y} \ell$ is calculated as follows:

$$\frac{\partial \ell}{\partial b_{y,i}} = \frac{\partial \ell}{\partial \tilde{y}_i} \frac{\partial \tilde{y}_i}{\partial b_{y,i}} = \frac{\partial \ell}{\partial \tilde{y}_i} = \hat{y}_i - y_i$$

Thus, the gradient can be expressed in vector form as:

$$\nabla_{b_y} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial b_{y,1}} \\ \vdots \\ \frac{\partial \ell}{\partial b_{y,n_y}} \end{bmatrix} = \begin{bmatrix} \hat{y}_1 - y_1 \\ \vdots \\ \hat{y}_{n_y} - y_{n_y} \end{bmatrix}$$

- ***Question 17:** Compute other gradients: $\nabla \tilde{h} \ell$, $\nabla W_h \ell$, $\nabla b_h \ell$.

To compute the gradient of the loss with respect to the intermediate output $\tilde{h}$, we use the chain rule:

$$\frac{\partial \ell}{\partial \tilde{h}_i} = \sum_k \frac{\partial \ell}{\partial h_k} \frac{\partial h_k}{\partial \tilde{h}_i}$$

Step 1: Compute $\frac{\partial h_k}{\partial \tilde{h}_i}$

The relationship between the intermediate output and the activation function is given by $h_k = \tanh(\tilde{h}_k)$. Thus, we have:

$$\frac{\partial h_k}{\partial \tilde{h}_i} = \frac{\partial \tanh(\tilde{h}_k)}{\partial \tilde{h}_i} = \begin{cases} 1 - \tanh^2(\tilde{h}_i) = 1 - h_i^2 & \text{if } k = i \\ 0 & \text{otherwise} \end{cases}$$

Step 2: Relate $\frac{\partial \ell}{\partial h_k}$ to Previous Derivations

We know from previous calculations that:

$$\frac{\partial \ell}{\partial \tilde{y}_i} = \hat{y}_i - y_i = \delta_i^y$$

Also, given $\tilde{y}_i = \sum_{j=1}^{n_h} W_{i,j}^y h_j + b_i^y$, we can express:

$$\frac{\partial \ell}{\partial h_k} = \sum_{j=1}^{n_y} \frac{\partial \ell}{\partial \tilde{y}_j} \frac{\partial \tilde{y}_j}{\partial h_k} = \sum_{j=1}^{n_y} \delta_j^y W_{j,k}^y$$

Step 3: Combine Results to Find $\frac{\partial \ell}{\partial \tilde{h}_i}$

Now we substitute our findings into the equation:

$$\frac{\partial \ell}{\partial \tilde{h}_i} = \sum_k \frac{\partial \ell}{\partial h_k} \frac{\partial h_k}{\partial \tilde{h}_i}$$

Substituting the computed terms:

$$= \sum_k \sum_j \delta_j^y W_{j,k}^y \times \begin{cases} 1 - h_i^2 & \text{if } k = i \\ 0 & \text{otherwise} \end{cases}$$

This simplifies to:

$$= (1 - h_i^2) \left(\sum_j \delta_j^y W_{j,i}^y \right) = \delta_i^h$$

Thus, the gradient $\nabla_{\tilde{h}} l$ is a vector where each element is given by δ_i^h :

$$\nabla_{\tilde{h}} l = (1 - h^2) \cdot (\nabla_{\tilde{y}} l * W^y)$$

Step 4: Compute Gradient with Respect to Weights W^h

Next, we find the gradient with respect to the weights W^h :

$$\nabla_{W^h} l = \sum_k \frac{\partial \ell}{\partial \tilde{h}_k} \frac{\partial \tilde{h}_k}{\partial W_{i,j}^h}$$

From our earlier work, we know:

$$\frac{\partial \ell}{\partial h_k} = (1 - h_k^2) \left(\sum_j \delta_j^y W_{j,k}^y \right) = \delta_k^h$$

We also compute:

$$\frac{\partial \tilde{h}_k}{\partial W_{i,j}^h} = \begin{cases} x_j & \text{if } k = i \\ 0 & \text{otherwise} \end{cases}$$

Combining these results yields:

$$\frac{\partial \ell}{\partial W_{i,j}^h} = \sum_k \delta_k^h \times \begin{cases} x_j & \text{if } k = i \\ 0 & \text{otherwise} \end{cases} = \delta_i^h x_j$$

Thus, we can express the gradient as:

$$\nabla_{W^h} l = \nabla_{\tilde{h}} l^T \cdot x$$

Step 5: Gradient with Respect to Biases b^h

Finally, we compute the gradient with respect to the biases:

$$\frac{\partial \ell}{\partial b_i^h} = \sum_k \frac{\partial \ell}{\partial \tilde{h}_k} \frac{\partial \tilde{h}_k}{\partial b_i^h} = \sum_k \delta_k^h \times \begin{cases} 1 & \text{if } k = i \\ 0 & \text{otherwise} \end{cases} = \delta_i^h$$

Thus, the gradient for the biases can be expressed as:

$$\nabla_{b^h} l = \nabla_{\tilde{h}} l$$

1.2 Implementation

1.2.1 Forward and backward manual

By Figure 1.1, in the training process of the "Global learning procedure by hands," the training accuracy significantly increases as the number of iterations grows, indicating that the model's fit to the training data improves steadily. Simultaneously, the training loss decreases, showing that the model reduces its prediction error during training. The test accuracy also rises in the early stages, but it lags behind the training accuracy, suggesting that the model's generalization to test data is relatively weaker. Additionally, while the test loss shows an overall decreasing trend, it declines more slowly compared to the training loss, further indicating limited generalization performance on the test data, potentially due to overfitting.

We can observe a large fluctuation in loss around epoch 100, which occurs for both the train and test sets. This may be caused by unstable gradient updates, a learning rate that is too high, a small batch size, or model overfitting. Therefore, we will investigate different learning rates and batch sizes to observe the results, and we will increase the number of training epochs, as we noticed that there are still minor fluctuations around epoch 150, indicating that the model might not have converged yet.

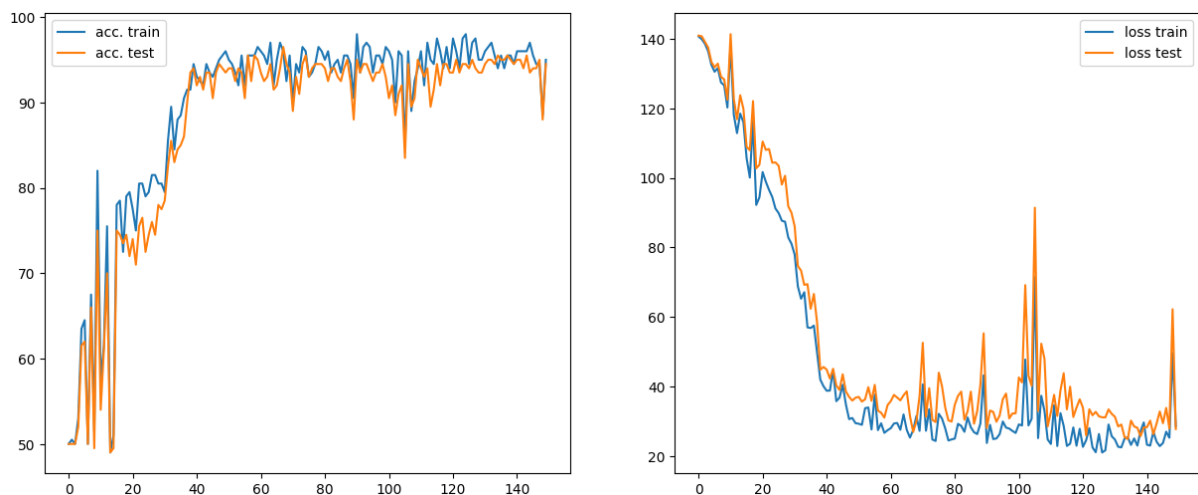


Figure 1.1: By Hands: Training and Testing Performance Progression Over Epochs

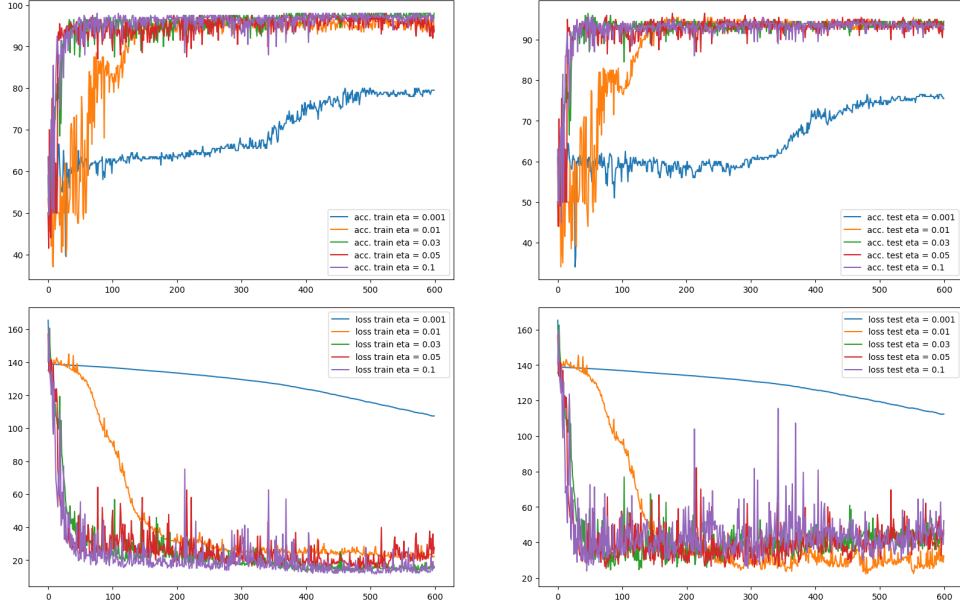


Figure 1.2: By Hands: Influence Of Learning Rate With Batch Size 10

By plot Figure 1.2, in our experiment on the influence of learning rate, we observed that a lower learning rate (e.g., 0.001) caused the training process to progress very slowly, with a gradual improvement in accuracy and a slow decrease in loss. While the process was stable, the convergence speed was quite slow. As the learning rate increased (e.g., 0.01 and 0.03), the accuracy of both the training and test sets improved more rapidly, and the loss decreased significantly, indicating a much more efficient learning process. Between these two, it was clear that a learning rate of 0.03 was optimal, as it resulted in the fastest convergence and the highest accuracy. However, when the learning rate was further increased to 0.1, we observed significant fluctuations in the loss curves for both the training and test sets, with the test accuracy and loss becoming unstable. This is because a learning rate that is too high causes overly aggressive parameter updates, potentially skipping over the optimal solution and hindering convergence.

Due to the significant differences in the curve fluctuations for different batch sizes, we did plot the small and large batch sizes separately. By Figure 1.3, for smaller batch sizes (such as 10 and 20), they update the model parameters more frequently, leading to a faster increase in accuracy. The accuracy curve is smoother with minimal fluctuations and converges quickly, and the loss curve is also more stable and decreases rapidly. For medium batch sizes (such as 50, 80, and 100), the curves fluctuate significantly, the accuracy improves slowly, the initial loss values are higher, and the convergence is slower, generally around 200 epochs. Before that, accuracy hovers between 50% and 80%, but the final accuracy is roughly on par with that of smaller batches, with only about a 3% difference. As for the very large batch size of 200, there is little to summarize, as the batch size is too large, resulting in infrequent updates of model parameters, leading to very poor performance.

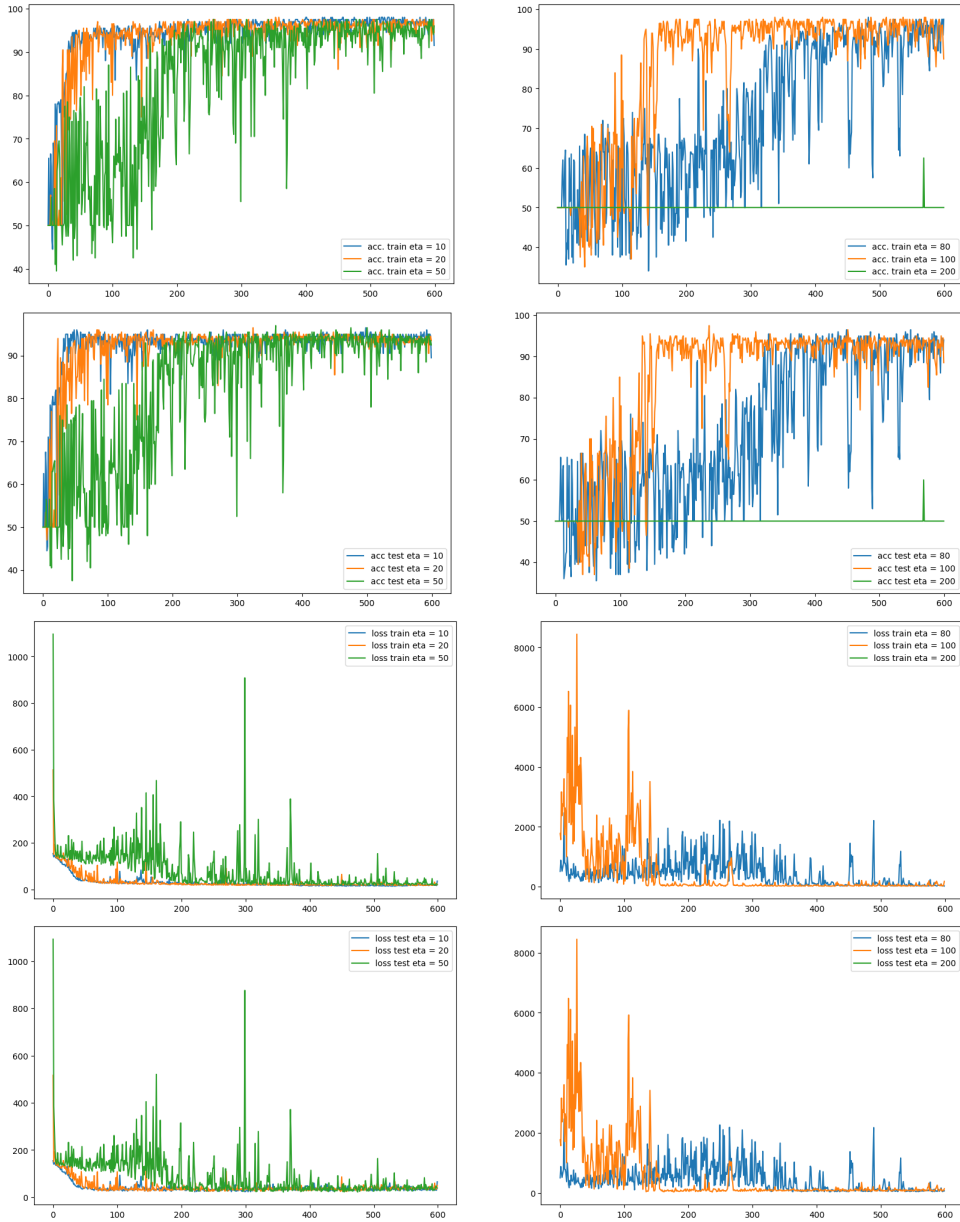


Figure 1.3: By Hands: Influence Of Batch Size With Learning Rate 0.03

1.2.2 Simplification of the backward pass with `torch.autograd`

Figure 1.4, Compared to manually calculating gradients, using `torch.autograd` resulted in much smoother and faster-converging accuracy and loss curves for both training and testing. The training accuracy increased more rapidly with significantly reduced fluctuations, and the test accuracy became more stable. The loss curve in the `autograd` section showed a more consistent and smooth decline with fewer fluctuations, whereas the manually calculated gradients exhibited larger fluctuations (especially sudden, significant swings) and a slower convergence rate. Overall, `autograd` significantly improved training efficiency and model stability, as it simplified the complex gradient computation and parameter update process, reducing the manual backpropagation workload and the likelihood of errors.

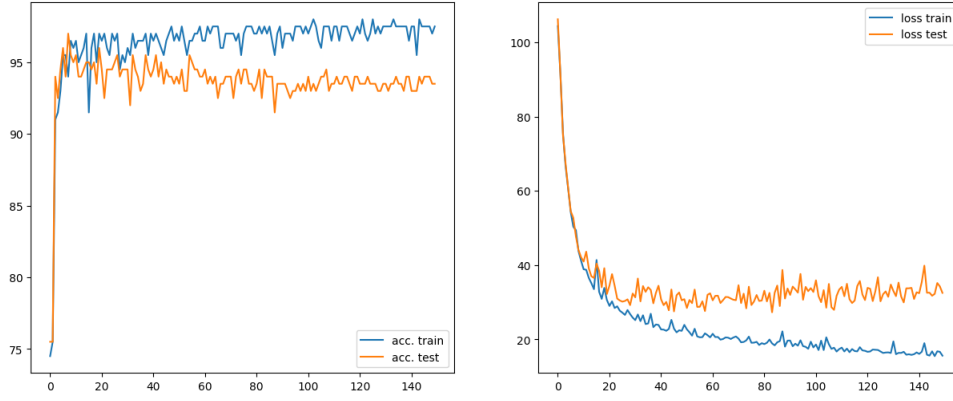
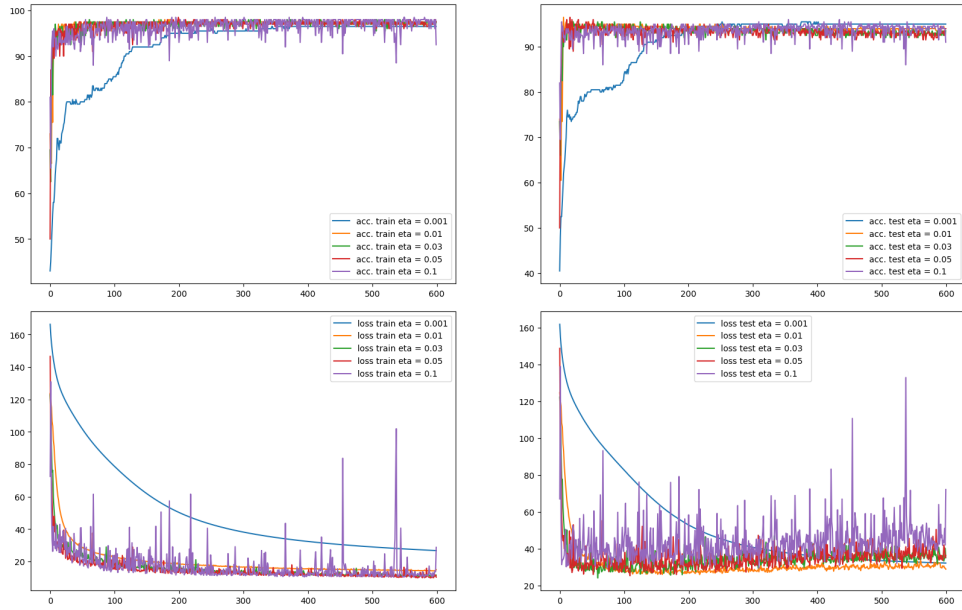
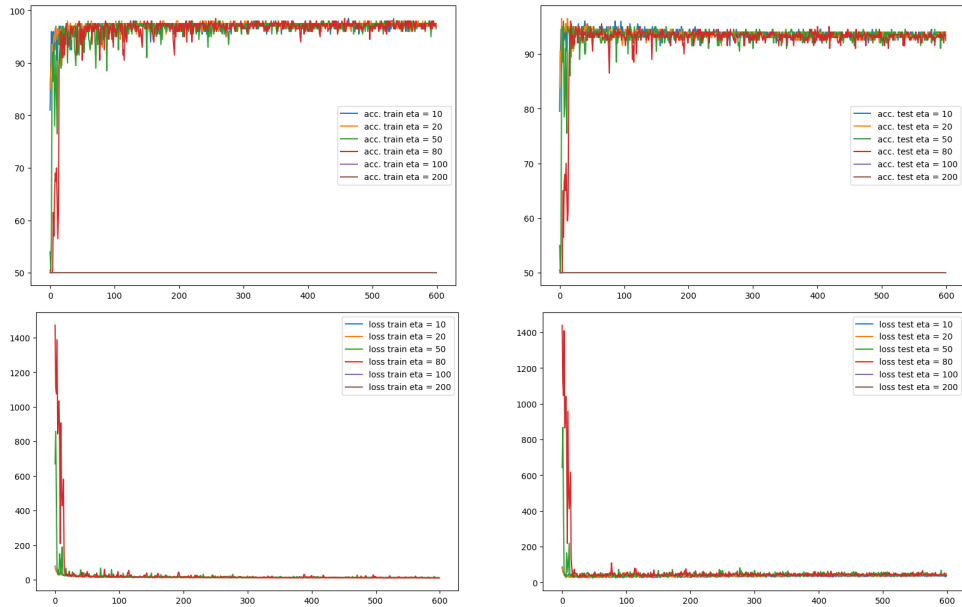


Figure 1.4: Torch.autograd: Training and Testing Performance Progression

Figure 1.5, With `torch.autograd`, the impact of learning rate and batch size showed significant improvements compared to the manual computation section. Smaller learning rates performed relatively smoothly in both sections, but in `autograd`, the model's convergence speed was faster, and higher learning rates produced more stable results with fewer fluctuations. Regarding batch size, smaller batches in `autograd` also performed better, with faster accuracy improvements and smoother loss reduction, and with less fluctuation compared to the manual section. Larger batch sizes performed relatively better in `autograd` as well; although the update frequency was lower, the curves were more stable and the convergence speed was faster. Overall, `autograd` improved the model's convergence speed and stability across different learning rates and batch sizes, showing clear advantages over manual computation.



(a) autograd: Influence Of Learning Rate With Batch Size 10

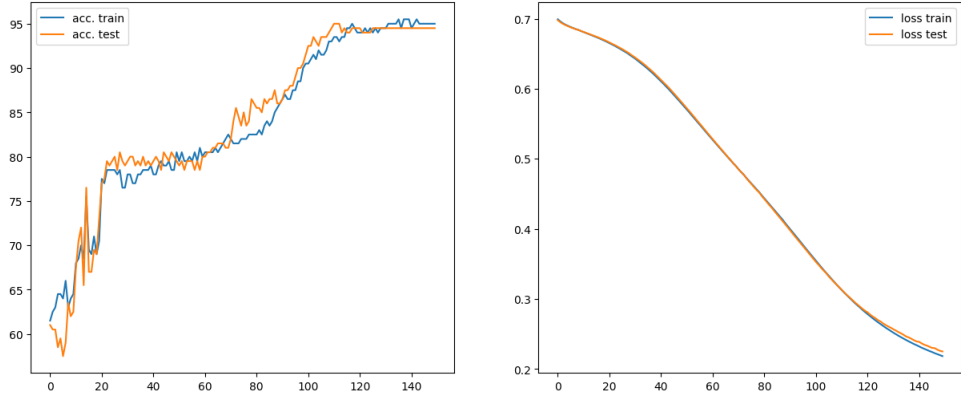


(b) autograd: Influence Of Batch Size With Learning Rate 0.03

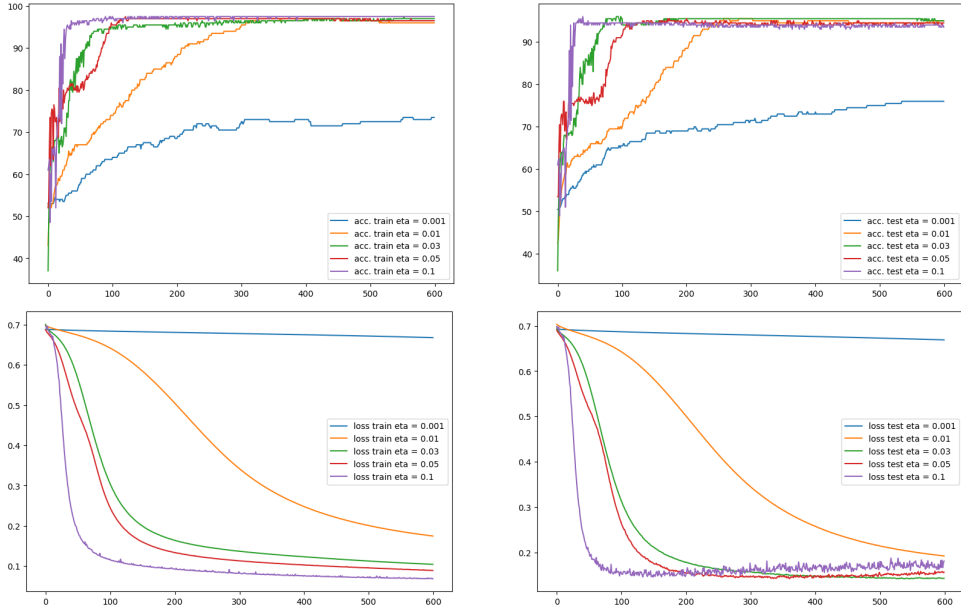
Figure 1.5: autograd: Accuracy and Loss Of the Influence of Learning Rate and Batch Size

1.2.3 Simplification of the forward pass with `torch.nn` layers

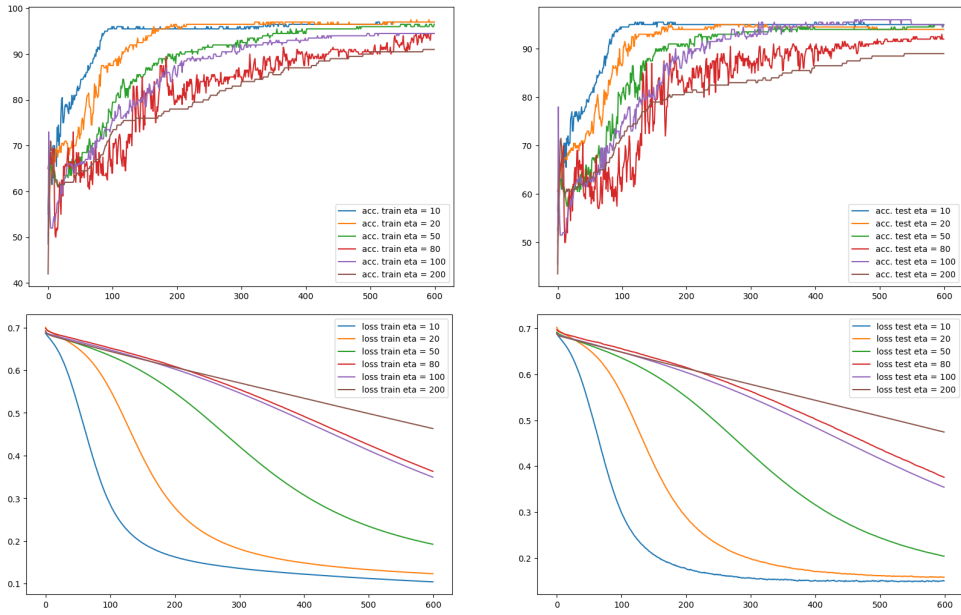
Figure 1.6, With `torch.nn` layers, the convergence speed of accuracy during the training and testing process slowed down compared to the manual and `torch.autograd` methods, with accuracy being similar to that of `torch.autograd`, but the loss was significantly reduced. The influence of learning rate and batch size showed a smoother training process. Smaller learning rates (e.g., 0.001) remained very stable, with accuracy hovering around 70%.



(a) torchNN: Training and Testing Performance Progression



(b) torchNN: Influence Of Learning Rate With Batch Size 10



(c) torchNN: Influence Of Batch Size With Learning Rate 0.03

Figure 1.6: Curves of torch.NN

However, higher learning rates (e.g., 0.03) performed better than in the manual and `torch.autograd` methods, with faster convergence and almost no fluctuations. In terms of batch size, smaller batches (e.g., 10 and 20) continued to perform excellently, with accuracy rapidly improving and the loss curve quickly decreasing. Larger batch sizes (e.g., 50 and 100) also exhibited greater stability, with significantly reduced fluctuations and faster convergence than the other two methods. The combination of a higher learning rate (e.g., 0.03) and smaller batch sizes (e.g., 10 and 20) not only accelerated convergence but also maintained minimal fluctuations. Overall, models trained with `torch.nn` layers outperformed the previous two methods in terms of performance and stability, with better generalization capabilities.

1.2.4 Simplification of SGD with `torch.optim`

Figure 1.8, With `torch.optim`, there is not much difference compared to the previous use of `torch.nn` layers, even the curve plots show almost no change, with only differences in convergence speed and the influence of learning rate and batch size is also similar too.

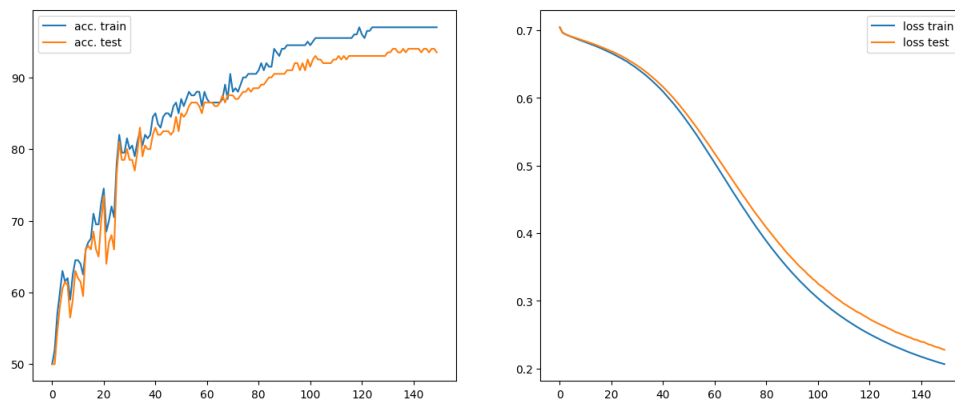
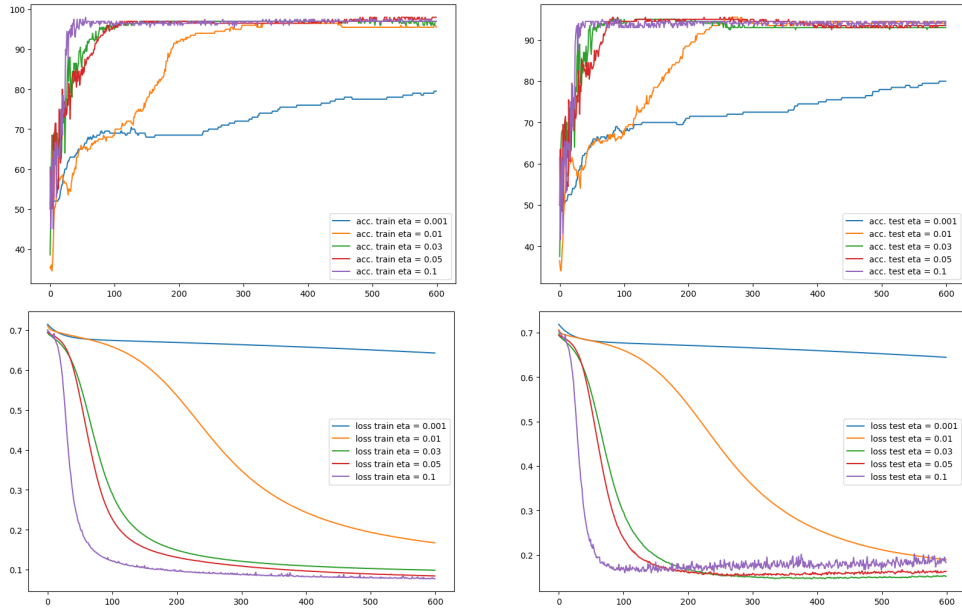
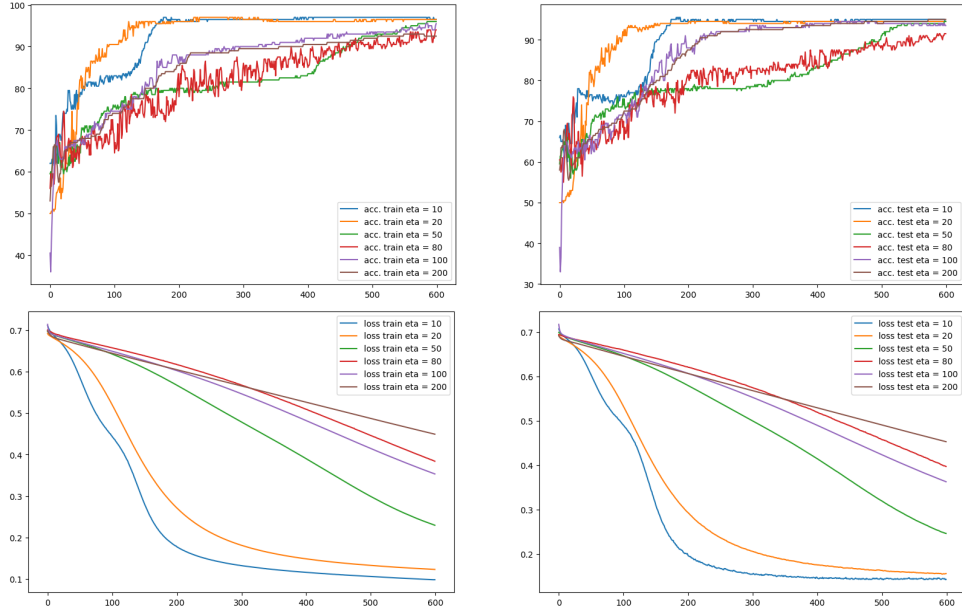


Figure 1.7: torchOptim: Training and Testing Performance Progression



(a) torchOptim: Influence Of Learning Rate With Batch Size 10

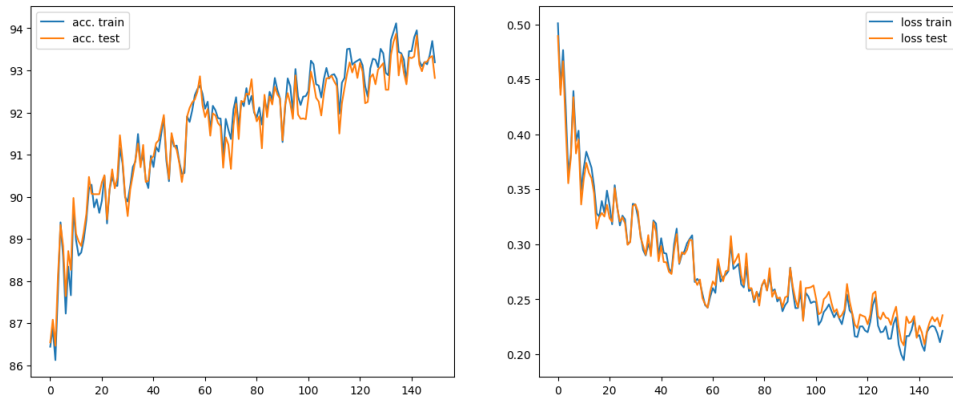


(b) torchOptim: Influence Of Batch Size With Learning Rate 0.03

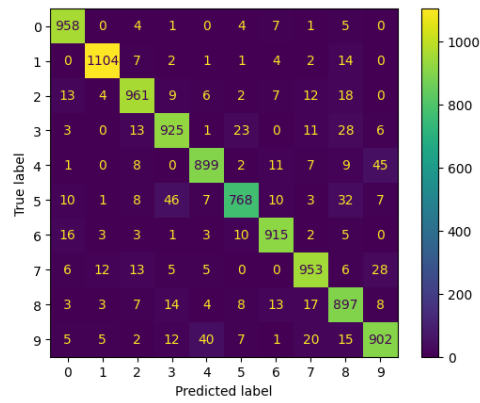
Figure 1.8: Curves of torch.optim

1.2.5 MNIST Application

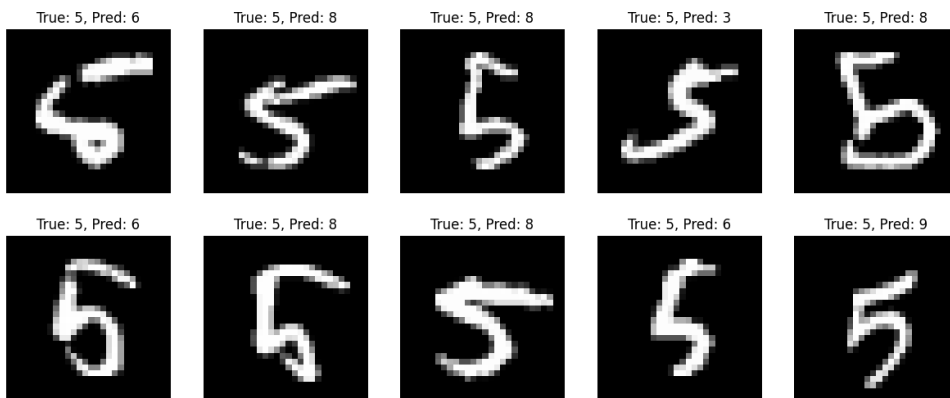
Applying the MNIST dataset with `torch.NN` and `torch.optim`, we achieved an accuracy of 93.6% on the training set and 93.5% on the test set. Through the confusion matrix, we identified which digits were more likely to be misclassified, with digit 5 having the lowest accuracy and being most frequently confused with digit 3. As shown in the Figure 1.9, we also display some correctly classified samples of digit 5 along with some misclassified samples.



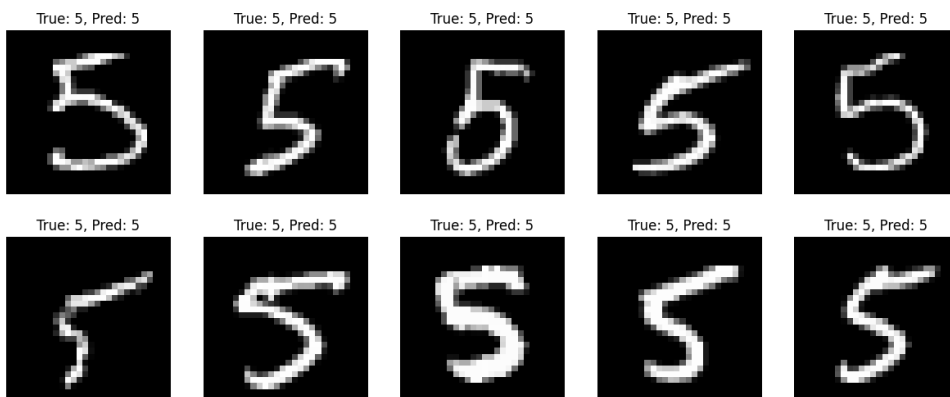
(a) Training and Testing Performance Progression Applying MNIST



(b) Confusion Matrix



(c) Example Of Misclassified With Class 5



(d) Example Of Well Classified With Class 5

Figure 1.9: MNIST

1.2.6 Bonus: SVM

The performance of the linear SVM is not good, primarily because a linear SVM can only use a straight line or a hyperplane in higher-dimensional space to separate two classes. However, our dataset consists of points distributed in a circular pattern, and it is not linearly separable.

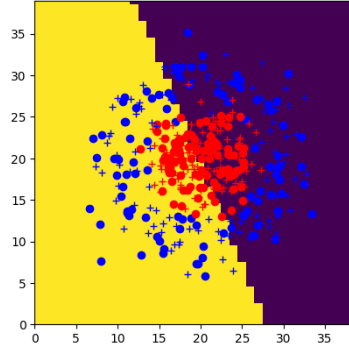


Figure 1.10: linear SVM: With Accuracy 53%

Next, we used four more complex SVM kernels. Among these kernels, the RBF kernel (Gaussian kernel) performed the best because it can map data to a higher-dimensional space, generating flexible nonlinear decision boundaries that adapt to the complex distribution of the data, particularly excelling in classifying the circular distribution present in this dataset.

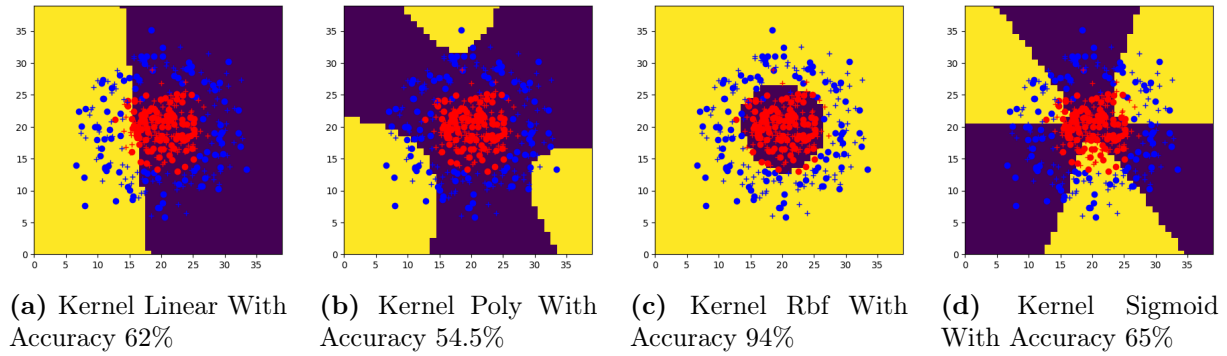
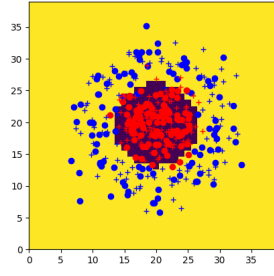
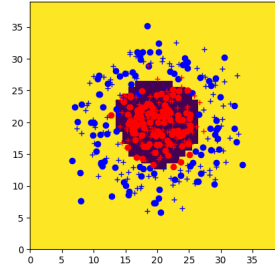


Figure 1.11: Four Different Kernels

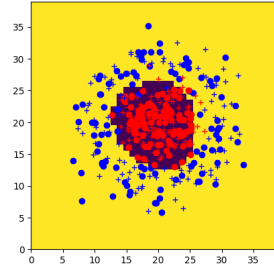
The regularization parameter C controls the tolerance of the SVM model for misclassifications in the training data. As the results show, a smaller C value tends to achieve slightly higher accuracy compared to a larger C value because a smaller C applies stronger regularization, allowing more misclassifications, which leads to smoother decision boundaries and improves the model's generalization ability, helping to avoid overfitting. On the other hand, with a larger C value, the accuracy tends to slightly decrease as C increases because the model becomes more strict in classifying the data, creating more complex decision boundaries, which may result in overfitting to the training data and thus reducing its ability to generalize to new data.



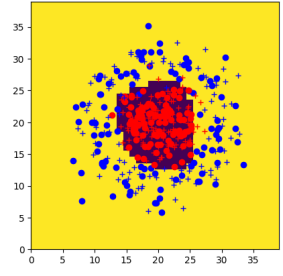
(a) $C=0.1$ With Accuracy 95%



(b) $C=1$ With Accuracy 94%



(c) $C=10$ With Accuracy 93.5%



(d) $C=100$ With Accuracy 93%

Figure 1.12: Four Different Parameter C Of Regularization

Chapter 2

Convolutional Neural Networks

2.1 Introduction to Convolutional Neural Networks

2.1.1 Convolutional layers

- **Question 1:** Considering a single convolution filter of padding p , stride s and kernel size k , for an input of size $x \times y \times z$, what will be the output size? How much weight is there to learn? How much weight would it have taken to learn if a fully-connected layer were to produce an output of the same size?

If we use a single convolution filter with padding p , stride s , and kernel size k on an input of size $x \times y \times z$, the output size will be:

$$\left(\left\lfloor \frac{x - k + 2p}{s} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{y - k + 2p}{s} \right\rfloor + 1 \right) \times z$$

The number of weights (parameters) the convolution layer needs to learn is $k \times k \times z$, where $k \times k$ is the filter size and z is the number of input channels. If we used a fully connected layer to get the same output size, it would need to learn a lot more weights, specifically

$$(x \times y \times z) \times (W_{\text{out}} \times H_{\text{out}}) + (W_{\text{out}} \times H_{\text{out}})$$

weights.

- ***Question 2:** What are the advantages of convolution over fully-connected layers? What is its main limit?

Convolutional layers are better than fully-connected layers because they use fewer parameters, making learning faster and more efficient. They keep the spatial structure of images, which helps in recognizing patterns, and can find features no matter where they are located. However, their main limit is that they struggle to see the big picture, making it hard to understand relationships between distant parts of an image

- ***Question 3:** Why do we use spatial pooling?

We use spatial pooling to reduce the size of feature maps while keeping important information. It simplifies the data, to make it easier for the network to process and help it become more robust to small shifts and distortions. Overall, pooling improves efficiency and reduces the risk of overfitting

- ***Question 4:** Suppose we try to compute the output of a classical convolutional network for an input image larger than the initially planned size (224x224 in the example). Can we use all or part of the layers of the network on this image?

Initially, convolutional layers can process larger images without any issues because they operate on local regions. However, when it comes to the fully connected layers (like the $1 \times 1 \times 4096$ layer), they expect a specific input size. If the input image is larger than what the network was designed for (like 224×224), the dimensions will not match. Therefore, while we can use the earlier layers of the network, we'll need to adjust or resize the image before reaching the fully connected layers.

- **Question 5:** Show that we can analyze fully-connected layers as particular convolutions.

We can analyze fully connected layers as particular convolutions by viewing the layer as a convolution with a kernel size equal to the input size. In a fully connected layer, each neuron connects to every input feature, similar to how a convolutional layer uses a filter that covers the entire input.

- **Question 6:** If fully-connected layers are replaced by their equivalent in convolutions, answer question 4 again. If we can calculate the output, what is its shape and interest?

If we replace fully connected layers with their convolutional equivalents, the feature map shape will change based on the input size. The advantage of this approach is that we can apply the same network architecture to images of different sizes without altering the model's parameters. Additionally, the network can capture features at various scales in larger images, which is beneficial for tasks such as object detection. By preserving spatial information in the output, we can better analyze and interpret features within the context of the original input image

- **Question 7:** What are the sizes of the receptive fields of the neurons of the first and second convolutional layers? What happens to the deeper layers? How do we interpret it?

The receptive field size of the first layer is equal to the kernel size, so for a 3×3 kernel, it is 3×3 . For the second layer, the receptive field depends on the kernel size and stride. With a 3×3 kernel and a stride of 1, the receptive field is 5×5 . If the stride is 2, it becomes 7×7 .

2.2 Training from scratch of the model

2.2.1 Network architecture

- **Question 8:** For convolutions, we want to keep the same spatial dimensions at the output as at the input. What padding and stride values are needed?

To maintain the output size x equal to the input size x in a convolutional layer, use the output size formula:

$$x' = \frac{x - k + 2p}{s} + 1$$

By setting the stride $s = 1$ and $x' = x$, and substituting into the formula, we derive the padding:

$$p = \frac{k - 1}{2}$$

Therefore, the condition to keep the output size the same as the input size is a stride $s = 1$ and padding $p = \frac{k-1}{2}$.

- **Question 9:** For max poolings, we want to reduce the spatial dimensions by a factor of 2. What padding and stride values are needed?

To halve the spatial dimensions in a max-pooling layer, the most common method is to use the following formula for output size:

$$x' = \frac{x - k + 2p}{s} + 1$$

By setting the pooling window size $k = 2$, stride $s = 2$, and padding $p = 0$, the output dimensions will be half of the input dimensions.

Therefore, the condition to halve the output size is a pooling window of $k = 2$, stride $s = 2$, and padding $p = 0$.

In fact, as long as the stride is $s = 2$ and padding is $p = 0$, the spatial dimensions will be reduced by half. The pooling window size can be larger, e.g., $k = 3 \times 3$ or $k = 4 \times 4$, and the output will still be halved as long as $s = 2$.

- ***Question 10:** For each layer, indicate the output size and the number of weights to learn. Comment on this distribution.

For each layer in the network, the output size and the number of learnable parameters are calculated as follows:

- **conv1:** Input size $32 \times 32 \times 3$, 32 filters of size 5×5 , stride $s = 1$, padding $p = 2$.
Output size: $32 \times 32 \times 32$

Number of weights: $32 \times (5 \times 5 \times 3) = 2400$

- **pool1**: Input size $32 \times 32 \times 32$, pooling window 2×2 , stride $s = 2$, no padding.
Output size: $16 \times 16 \times 32$
Number of weights: none (pooling layers do not have learnable parameters)
- **conv2**: Input size $16 \times 16 \times 32$, 64 filters of size 5×5 , stride $s = 1$, padding $p = 2$.
Output size: $16 \times 16 \times 64$
Number of weights: $64 \times (5 \times 5 \times 32) = 51200$
- **pool2**: Input size $16 \times 16 \times 64$, pooling window 2×2 , stride $s = 2$, no padding.
Output size: $8 \times 8 \times 64$
- **conv3**: Input size $8 \times 8 \times 64$, 64 filters of size 5×5 , stride $s = 1$, padding $p = 2$.
Output size: $8 \times 8 \times 64$
Number of weights: $64 \times (5 \times 5 \times 64) = 102400$
- **pool3**: Input size $8 \times 8 \times 64$, pooling window 2×2 , stride $s = 2$, no padding.
Output size: $4 \times 4 \times 64$
- **fc4**: Input size $4 \times 4 \times 64 = 1024$ (flattened), fully connected to 1000 neurons.
Output size: 1000
Number of weights: $1024 \times 1000 = 1024000$
- **fc5**: Input size 1000, fully connected to 10 neurons (output layer).
Output size: 10
Number of weights: $1000 \times 10 = 10000$

This distribution of parameters shows that the fully connected layers contribute the most to the parameter count, while the convolutional layers have relatively fewer parameters due to weight sharing.

- **Question 11:** What is the total number of weights to learn? Compare that to the number of examples.

By the question before, the total number of learnable weights in the network is calculated as:

$$2400 + 51200 + 102400 + 1024000 + 10000 = 1186000$$

Thus, the total number of parameters in the network is 1,186,000.

Comparing this with the number of training samples in the CIFAR-10 dataset:

- CIFAR-10 training set size: 50,000 images
- Each image size: $32 \times 32 \times 3 = 3072$ pixels

The total number of learnable parameters (1,186,000) is significantly larger than

the number of training images (50,000). We could observe that the model has relatively large parameters compared to the dataset, which may cause the model to overfit. Regularization techniques can be used to mitigate this problem.

- **Question 12:** Compare the number of parameters to learn with that of the BoW and SVM approach.

Among the three models, CNN has the largest number of parameters, as we calculated in the previous question, with 1,186,000 learnable parameters. The number of parameters in BoW (Bag of Words) depends on the dimensionality of the feature vectors, while the number of parameters in SVM (Support Vector Machine) also depends on the feature vector dimensions. For CIFAR-10, each image is flattened into a vector of $32 \times 32 \times 3 = 3072$ dimensions.

2.2.2 Network learning

- ***Question 14:** What is the major difference in calculating loss and accuracy in training versus testing in the provided code?

The test loss is calculated by averaging the losses of all test batches across each epoch. In contrast, the training loss is also averaged but reported for each individual batch. During the test phase, the model's weights are not updated after each batch, as there is no backpropagation. This approach prevents overfitting and allows for a true assessment of the model's generalization to unseen data. The primary goal of testing is to evaluate performance on new data, while training focuses on optimizing the model's parameters.

- ***Question 16:** What are the effects of the learning rate and batch-size?

The learning rate and batch size are essential hyperparameters that greatly influence training performance, speed, and stability.

A high learning rate can make the model overshoot the minimum, leading to instability and ineffective learning. Conversely, a low learning rate can slow down training significantly, potentially causing the model to settle on suboptimal solutions too soon. However, a lower learning rate can also help the model converge more accurately by allowing it to thoroughly explore the parameter space. We should aim for a balanced learning rate that's neither too high nor too low to ensure efficient training and precise convergence.

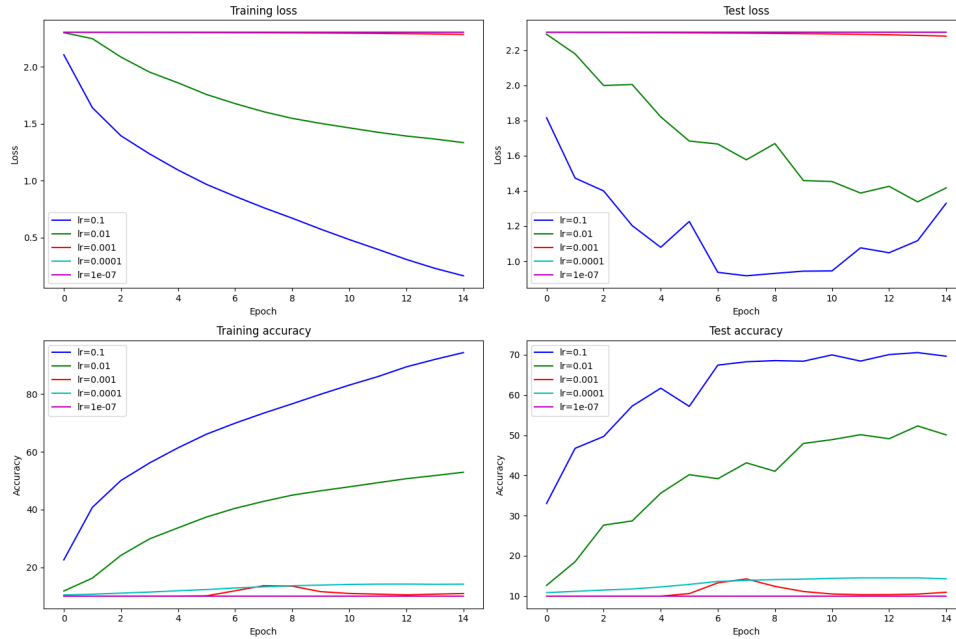


Figure 2.1: By Hands: Influence Of Learning Rate With Batch Size 128

A larger batch size gives a more accurate estimate of the gradient, which can result in more stable and consistent training. However, it also requires more memory. On the other hand, smaller batch sizes can add noise to the training process, making convergence slower and less straightforward. But they use less memory, making them a better choice for systems with limited resources.

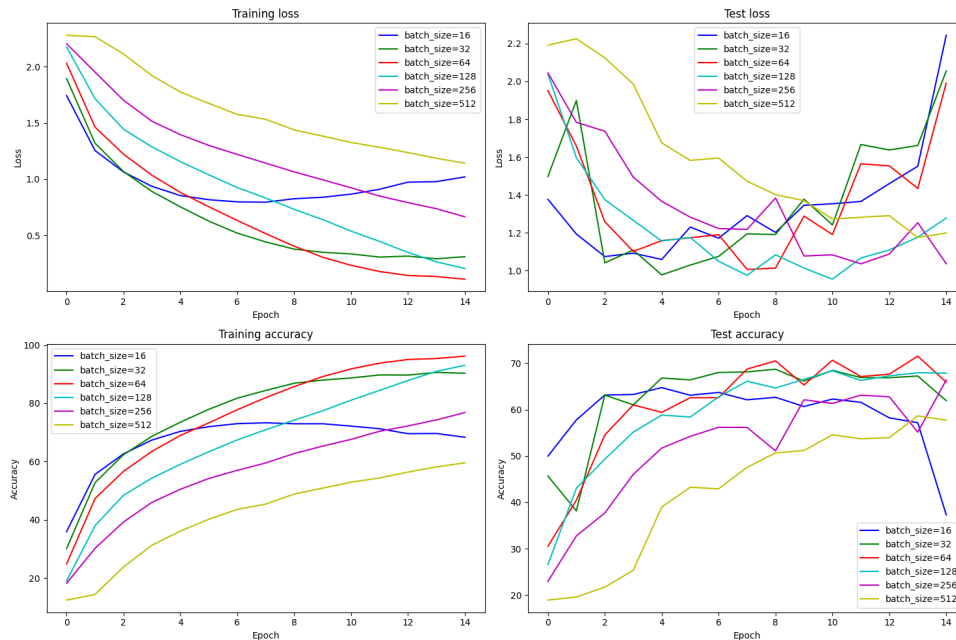


Figure 2.2: By Hands: Influence Of Batch Size With learning rate 0.1

- **Question 17:** What is the error at the start of the first epoch, in training and testing? How can you interpret this?

At the start of the first epoch, the model's weights are typically initialized randomly, which means it hasn't yet learned anything. For a dataset like CIFAR-10, which has 10 classes, the initial accuracy (precision at one or top-1 accuracy) is expected to be around 10%, since the model's performance is similar to random guessing.

- ***Question 18:** Interpret the results. What is wrong? What is this phenomenon?

At the end of training, as shown in the following figure, the training loss stabilizes close to zero, but the test loss starts increasing after 20 epochs. Similarly, while the training accuracy remains high, the test accuracy drops around 30 epochs. This behavior indicates overfitting, where the model performs well on the training data but struggles to generalize to unseen test data.

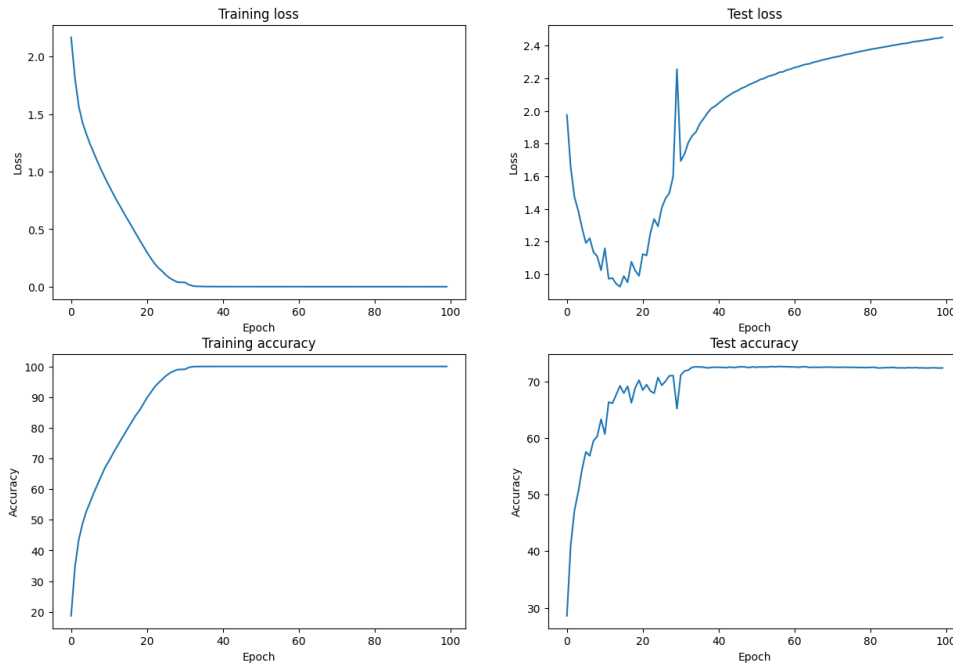


Figure 2.3: By Hands: batch size = 32, lr = 0.01

2.3 Results improvements

2.3.1 Standardization of examples

- **Question 19:** Describe your experimental results.

Standardization plays a key role in addressing the challenges posed by varying scales and ranges in raw data. By applying standardization, we observed improvements in model training efficiency, more consistent data distribution, and reduced numerical instability. Our experimental results demonstrate that standardizing the data not only accelerates the training process but also stabilizes the

model's performance. For instance, by epoch 20, the model had already surpassed 75% accuracy, requiring fewer epochs to reach convergence.

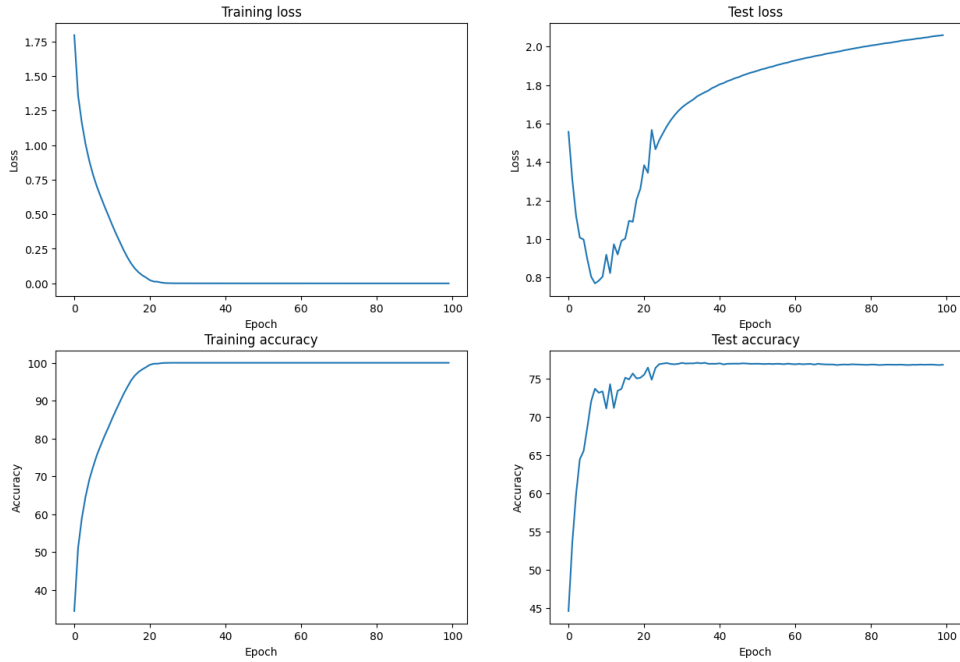


Figure 2.4: Impact of standardization on model training (batch size=32, lr = 0.01).

- **Question 20:** Why calculate the average image on the training examples and normalize the validation examples with the same image?

We calculate the average image on the training examples and normalize the validation examples with the same image to ensure that the validation data is transformed in the same way as the training data. This guarantees that the model evaluates its performance on data processed similarly to the training data, without prior knowledge of the validation set's specific characteristics. This approach prevents “data leakage” and ensures consistency, providing an impartial evaluation of the model's ability to generalize to new, unseen data.

2.3.2 Increase in training examples by data augmentation

- **Question 22:** Describe your experimental results and compare them to previous results.

By increasing dataset variability with random crops and flips, the model not only generalizes better and becomes more robust to real-world variations, but also achieves faster accuracy improvements. For example, at epoch 20, the training accuracy is noticeably better compared to previous results, demonstrating the positive impact of data augmentation on model performance.

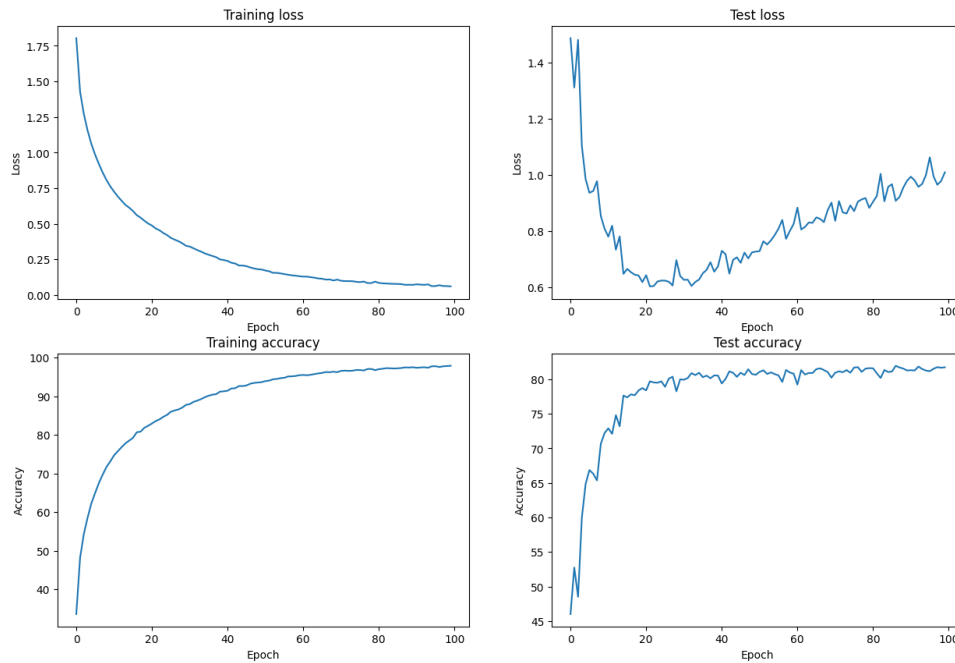


Figure 2.5: Impact of data augmentation on model training (batch size = 32, lr = 0.01)

- **Question 23:** Does horizontal symmetry work well on all types of images? Why or why not?

For many general tasks, such as those involving landscapes or objects like furniture, horizontal flipping doesn't impact the identity of the subject a chair or tree remains recognizable whether flipped or not. This makes flipping useful for improving model robustness. However, in tasks where orientation is essential, flipping can cause issues. For instance, in traffic sign recognition, flipping a "right turn" sign would incorrectly indicate a "left turn".

- **Question 24:** What are the limits of this type of data augmentation?

Using horizontal symmetry in data augmentation has its limits. First, it can lead to confusion in tasks where orientation matters and sometimes flipping an image might change its meaning or context.

Second, applying horizontal flips uniformly can worsen class imbalances if certain classes are already overrepresented. For example, flipping images of a left-handed person can mislead the model about their handedness.

Lastly, while horizontal flipping can create variability, it doesn't add new information beyond what's already in the data. This might restrict the model's ability to generalize to situations it hasn't seen before, limiting its effectiveness in real-world applications.

- **Bonus Question 25:** Explore other data augmentation methods and test some.

The data augmentations applied in our experiments are as follows:

- **Random Rotation:** `transforms.RandomRotation(15)` - This transformation randomly rotates the image by an angle of up to 15 degrees.
- **Color Jitter:** `transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)` - This transformation adjusts the brightness, contrast, saturation, and hue of the image randomly within specified ranges.

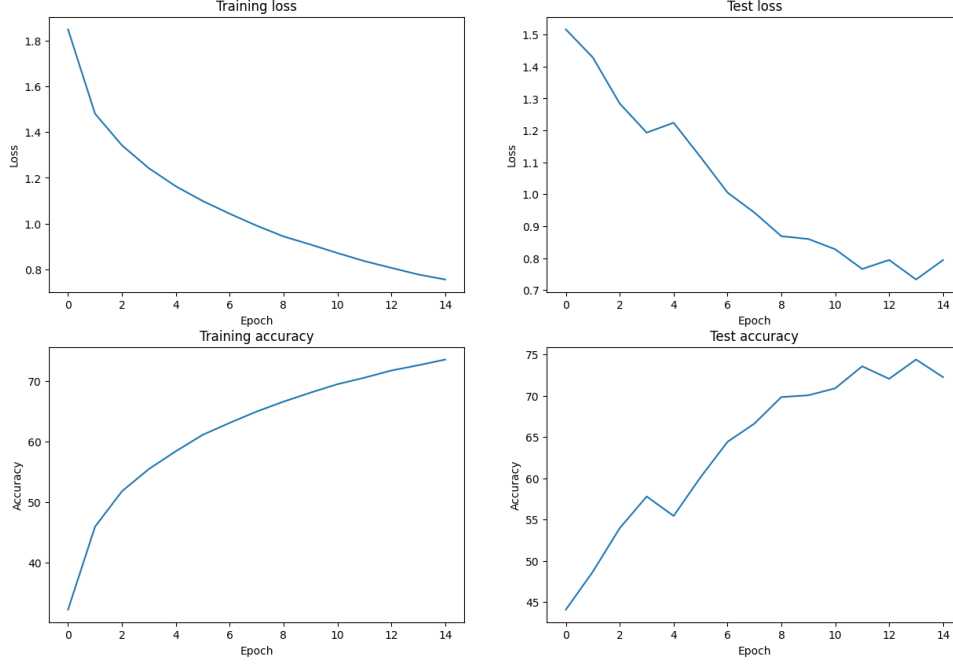


Figure 2.6: Random Rotation with 15 deg

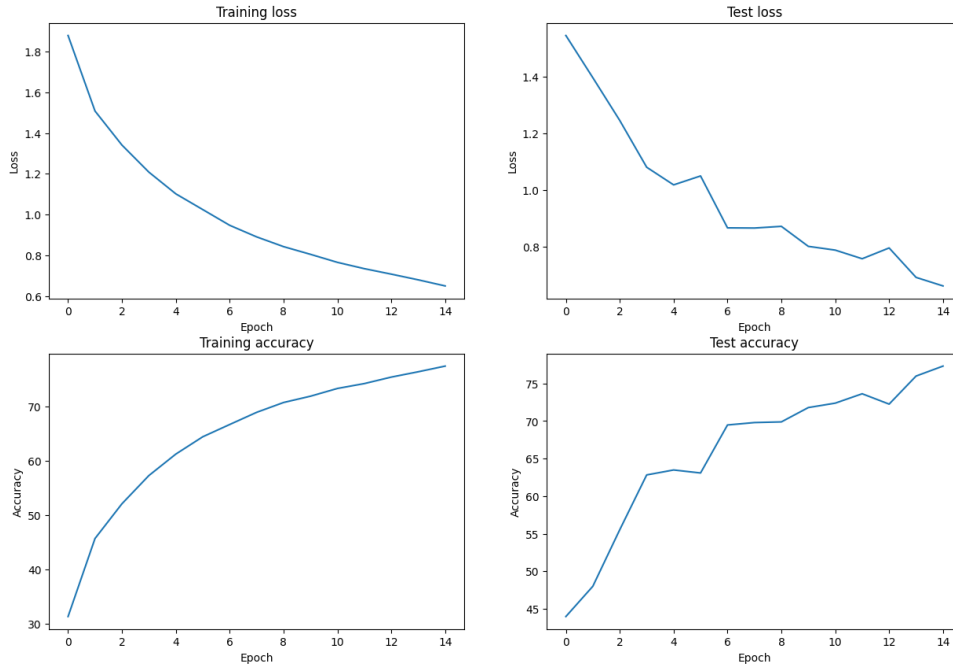


Figure 2.7: Color augmentation: (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)

Using Random Rotation as a data augmentation technique enhances the model's ability to recognize objects from various angles, making it more robust to real-world scenarios where orientations can vary. This helps improve generalization and reduces sensitivity to the exact positioning of objects in images. Meanwhile, Color Jitter increases the model's resilience to changes in lighting and color conditions by randomly adjusting brightness, contrast, saturation, and hue. This encourages the model to focus on essential features rather than being overly influenced by specific color variations, ultimately leading to improved performance across diverse environments.

2.3.3 Variants on the optimization algorithm

- **Question 26:** Describe your experimental results and compare them to previous results, including learning stability.

With the learning rate scheduler, the training loss drops more quickly and reaches near-zero early on, while the test loss increases steadily after an initial drop, indicating stronger overfitting. Training accuracy rapidly reaches 100%, but the test accuracy plateaus around 75%. This suggests that while the scheduler accelerates learning on the training data, it does not improve generalization and overfitting remains an issue.

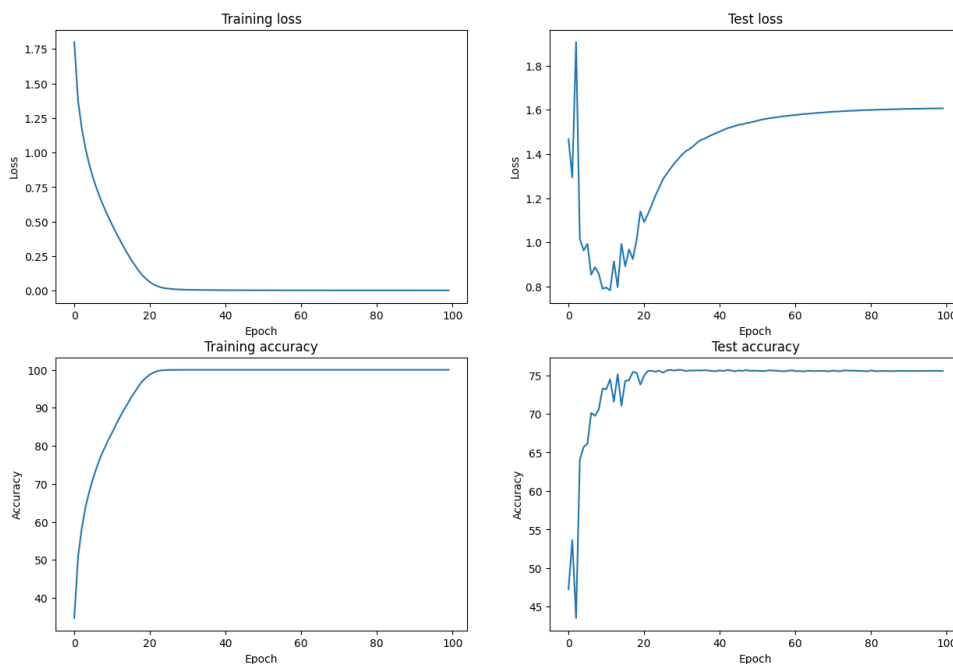


Figure 2.8: Variants on the optimization algorithm

- ***Question 27:** Why does this method improve learning?

The ExponentialLR learning rate scheduler improves learning by adjusting the learning rate throughout the training process. In the beginning, a higher learning rate allows the model to explore the parameter space quickly, helping it find better

solutions. As training progresses, the learning rate gradually decreases, which helps the model make smaller, more precise updates to fine-tune its parameters. This approach is beneficial because the loss function landscape can vary—some areas may require larger steps, while others need smaller steps for better accuracy. However, it's important to set the initial learning rate and decay factor carefully, as choosing them incorrectly can affect the training's effectiveness, either by reducing the learning rate too quickly or keeping it too high.

- **Bonus Question 28:** Explore other variants of SGD and learning rate strategies.

This question we've done in the notebook but the results were not that good, we didn't find a better way to solve it, so we don't put the plots and results here, but if you're interested in our "bad results", could check our notebook :/.

2.3.4 Regularization of the network by dropout

- **Question 29:** Describe your experimental results and compare them to previous results.

Integrating a dropout layer into a neural network is a smart way to prevent overfitting, especially before fully connected layers with many weights. As expected, this helped reduce overfitting and improved overall performance. The best results were achieved by combining dropout with a learning rate scheduler, showing how effective these techniques are together.

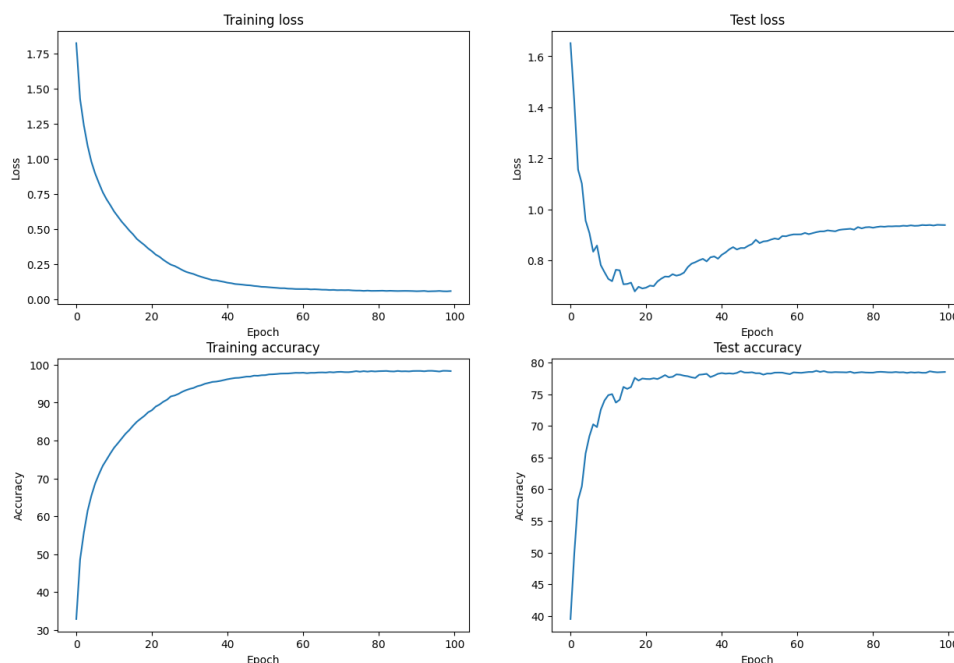


Figure 2.9: Regularization of the network by dropout (batch size =32, lr= 0.01)

- **Question 30:** What is regularization in general?

Regularization is a set of techniques used to prevent overfitting, which happens when a model learns the training data too well but struggles with new, unseen data. By adding constraints or penalties to the model during training, regularization helps the model find a good balance between fitting the training data and generalizing well to new data. Common methods include L1 and L2 regularization, dropout, data augmentation, and early stopping. These techniques keep the model from becoming too complex and help it perform better on different datasets.

- ***Question 31:** Discuss possible interpretations of dropout on the network behavior.

Dropout is a regularization technique that helps prevent overfitting by randomly turning off some neurons (and their connections) during training. This forces the remaining neurons to work harder and learn more independent features, making the model more robust. It also prevents neurons from becoming too dependent on each other, which can happen if they always rely on the same inputs.

Dropout can be seen as creating many smaller networks within the main network, each using different sets of neurons. During testing, all neurons are active, but their outputs are scaled to account for the dropout used in training. This approach improves the model's ability to generalize and make accurate predictions on new data.

- **Question 32:** What is the influence of the dropout hyperparameter?

A high dropout rate means more neurons are deactivated, increasing regularization and improving generalization. However, if the rate is too high, the model might struggle to learn and could underfit, losing too much information. A lower dropout rate keeps more neurons active, but this can lead to overfitting, especially in complex networks. In our case, using a low dropout rate improves training but causes overfitting, while a higher rate reduces overfitting significantly. If the rate is set to 1 (dropping all neurons), the model can't learn at all.

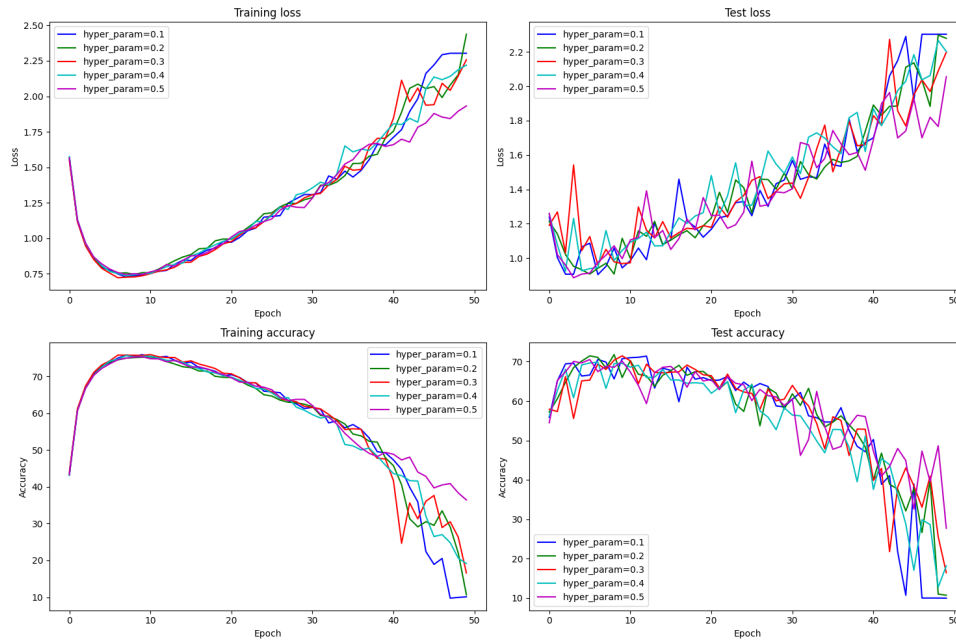


Figure 2.10: Influence of dropout rate p

- **Question 33:** What is the difference in the behavior of the dropout layer between training and testing?

During training, dropout randomly turns off some neurons with each batch of data, based on the dropout rate. This helps prevent the model from relying too much on specific neurons. During testing, dropout is turned off, meaning all neurons are active. This ensures that the model makes consistent and reliable predictions on new data.

2.3.5 Use of batch normalization

- **Question 34:** Describe your experimental results and compare them to previous results.

The training loss in this batch normalization case steadily decreases and reaches near zero, similar to the previous cases. However, the key difference is in how the accuracy converges and stabilizes much earlier. There is also much less fluctuation in the training accuracy curve, making the model's learning process smoother and more stable.

The test loss is lower overall in this run, and it shows fewer fluctuations compared to the previous results without batch normalization. It doesn't increase as sharply, indicating that the model is generalizing better. Test accuracy also improves more quickly and remains stable, without the sharp drops seen previously, which indicates reduced overfitting.

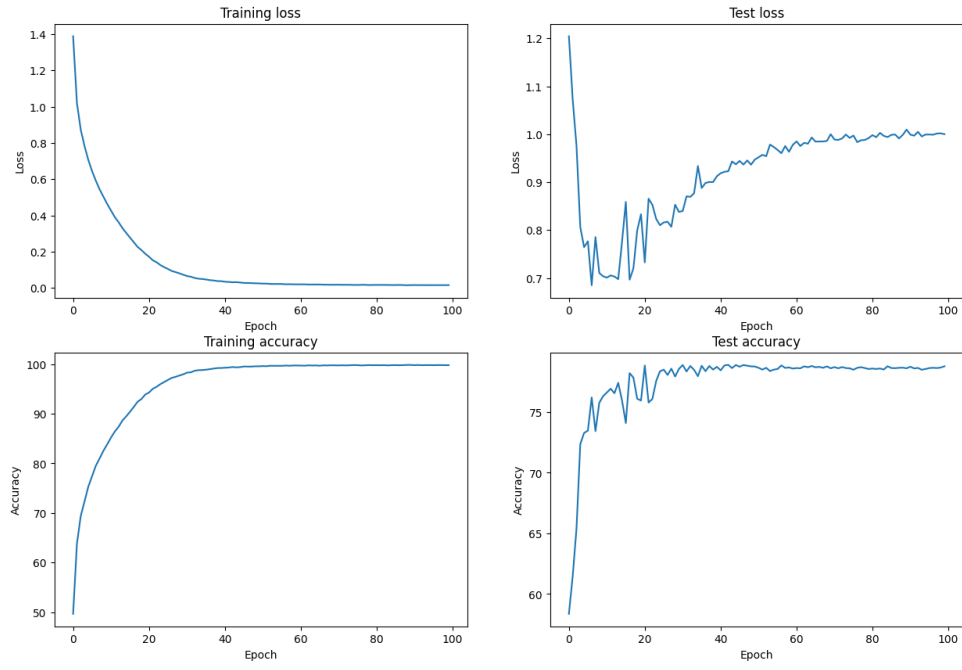


Figure 2.11: Regularization of the network by dropout (batch size =32, lr= 0.01

Chapter 3

Transformers

3.1 Self-Attention

- **Question:** What is the main feature of self-attention, especially compared to its convolutional counterpart. What is its main challenge in terms of computation/memory?

Self-Attention's core feature lies in its flexible long-range dependency modeling capability, enabling it to capture global relationships between any two positions in the input sequence (such as an image or sentence), distinguishing it from convolutional operations that only capture local features. Specifically, Self-Attention dynamically computes the similarity between each element and every other element, assigning weights so that each element can attend to the entire input in a single operation. This makes Self-Attention particularly useful in tasks like natural language processing and image processing, where capturing long-range dependencies is essential.

However, this global modeling capability also introduces significant computational and memory challenges: the complexity of Self-Attention grows quadratically with the input size, $O(N^2)$, as it requires computing the similarity between every pair of elements and generating an $N \times N$ attention matrix that holds the weights for each element pair. Consequently, as the sequence length or image resolution increases, the computational load and memory demands of Self-Attention increase rapidly, potentially leading to out-of-memory (OOM) issues. In our subsequent tests, we encountered memory exhaustion issues.

- **Question:** At first, we are going to only consider the simple case of one head. Write the equations.

Self-Attention with a single head can be broken down into three main steps:

1. Compute Queries, Keys, and Values:

Given input $X \in \mathbb{R}^{N \times d_{\text{model}}}$, we project it into Query Q , Key K , and Value V

matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learnable weights.

2. Calculate Attention Weights:

We compute the scaled dot-product attention weights A by taking the dot product of Q and K , then applying softmax:

$$A = \text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

where $A \in \mathbb{R}^{N \times N}$ gives attention weights for each pair of input positions.

3. Compute Weighted Sum for Attention Output:

The output of the Self-Attention mechanism, denoted Z , is obtained by multiplying the attention weights A with the Value matrix V :

$$Z = AV$$

where $Z \in \mathbb{R}^{N \times d_k}$ represents the weighted sum of values according to the attention distribution.

4. Apply Final Linear Projection:

Finally, we apply a linear projection $W \in \mathbb{R}^{d_k \times d_{\text{model}}}$ to map Z back to the original model dimension:

$$\text{Output} = ZW$$

This final projection ensures the output is compatible with the next layer in the model.

3.2 Multi-head self-attention

- Write the equations.

Multi-Head Self-Attention can be broken down into the following steps:

1. Compute Queries, Keys, and Values for Each Head:

Like the question before, it's the same way to compute the Query, Key and Value, just we need calculate each head.

For each attention head i :

$$Q_i = XW_{Q_i}, \quad K_i = XW_{K_i}, \quad V_i = XW_{V_i}$$

where $W_{Q_i}, W_{K_i}, W_{V_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learnable projection matrices for head i . Here, $d_k = \frac{d_{\text{model}}}{h}$, with h being the total number of heads.

2. Calculate Attention Weights and Output for Each Head: For each head i , we

calculate the scaled dot-product attention weights A_i and the output Z_i :

$$A_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right)$$

$$\text{head}_i = A_i V_i$$

where $A_i \in \mathbb{R}^{N \times N}$ is the attention weight matrix for head i , and $\text{head}_i \in \mathbb{R}^{N \times d_k}$.

3. **Concatenate and Project the Outputs:** After obtaining the output Z_i from each head, we concatenate them along the depth dimension and apply a final linear projection W to obtain the final Multi-Head Self-Attention output:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W$$

where $W \in \mathbb{R}^{(h \cdot d_k) \times d_{\text{model}}}$ is a learnable projection matrix, and the output matches the input dimension.

3.3 Transformer block

- **Write the equations.**

From the Transformer Block image in the notebook, we could divide it into two parts, first part is layer norm + Multi-Head Attention, second part is layer norm + MLP, the processes consisted several key steps, described below:

1. **Normalize the Input:** First, the input x is normalized to stabilize feature values, making them more suitable for further processing:

$$\text{Norm_x} = \text{Norm}(x)$$

2. **Generate Queries, Keys, and Values:** Using the normalized Norm_x , we compute the Query Q , Key K , and Value V matrices through separate linear transformations. These are used to calculate attention weights:

$$Q = \text{Norm}_x \cdot W_q, \quad K = \text{Norm}_x \cdot W_k, \quad V = \text{Norm}_x \cdot W_v$$

3. **Multi-Head Attention:** The Query, Key, and Value matrices are passed through the multi-head attention mechanism, calculating relationships between features and producing an attention output:

$$\text{MultiHeadOutput} = \text{MultiHead}(Q, K, V)$$

4. **Residual Connection and Normalization:** The attention output is added to the original input Norm_x (residual connection) and normalized again for stability:

$$\text{FirstPartOutput} = \text{Norm_x} + \text{MultiHeadOutput}$$

$$\text{Norm_h} = \text{Norm}(\text{FirstPartOutput})$$

5. Multi-Layer Perceptron (MLP): The intermediate output is passed through a two-layer fully connected network with GeLU activation in the first layer, enhancing the model's non-linear capabilities:

$$\text{MLPOutput}(x) = \text{GeLU}(\text{Norm_h} \cdot W_1 + b_1) \cdot W_2 + b_2$$

6. Final Output: The result of the MLP is added to the previous layer output to form the final output of this Transformer layer, ensuring stability and rich feature representation:

$$\text{Output} = \text{MLPOutput} + \text{Norm_h}$$

3.4 Full ViT model

- **Explain what is a Class token and why we use it?**

The Class Token is an additional vector along with the image patches, serves as input to the ViT. This token typically starts as a learnable embedding. In the input layer of ViT, the image is divided into multiple small patches, where each patch is treated as a token. A single unique Class Token is added to the sequence of these patch tokens, and based on the structure, it is placed at the beginning of the sequence.

The primary role of the Class Token is to aggregate global information and simplify the final classification process. In ViT, all patches along with the Class Token, are processed through multiple layers of the Transformer encoder. Due to the self-attention mechanism of the Transformer, each patch can interact with every other patch. As processing progresses through each layer, the Class Token accumulates and integrates more features from different patches, effectively aggregating information from the entire image. In the output layer of the Transformer, we directly use the embedding of the Class Token for classification, eliminating the need for additional global pooling operations, such as global average pooling. This approach simplifies the final classification process.

- **Explain whay is the the positional embedding (PE) and why it is important?**

PE is a set of vectors used to represent the positions (or order) within an input sequence. In ViT, the input image is divided into multiple patches, and encoded as tokens. However, the self-attention mechanism cannot inherently distinguish the specific positions of these tokens within the image. PE assigns a unique vector to each patch token, indicating its relative position in the image. These PE are then concatenated with both the patch tokens and the class token to form the final input.

The importance of PE lies in its ability to introduce positional information, allowing ViT to understand the relative location of each patch within the image. This enables the model to grasp the spatial layout and order of the image, preventing the loss of structural information.

3.5 Experiment on MNIST

3.5.1 Quick function

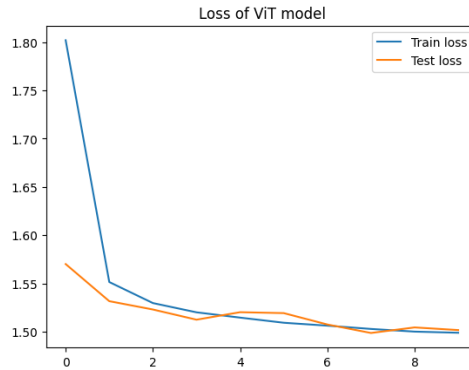


Figure 3.1: Embed_dim=32, patch_size=7, nb_blocks=2

3.5.2 Experimenting embed_dim

The embedding dimension represents the size of the vector space in which each patch token is embedded. From the plot Figure 3.2, we observe that increasing embed_dim (from 16 to 32 to 64) generally results in lower training and test losses. This suggests that a higher embedding dimension allows ViT to capture more complex patterns and subtle information from the image, enhancing performance. However, higher dimensions also increase computational costs, as larger embeddings require more memory and processing power.

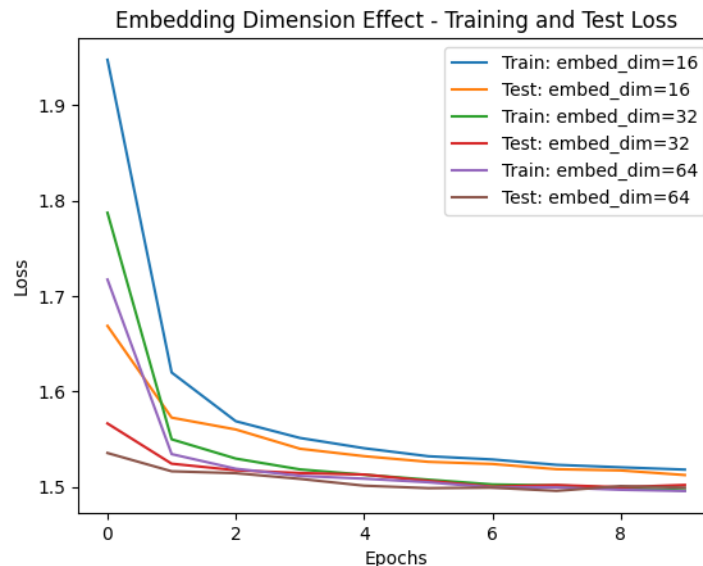


Figure 3.2: Effects Of Embedding Dimension

3.5.3 Experimenting patch_size

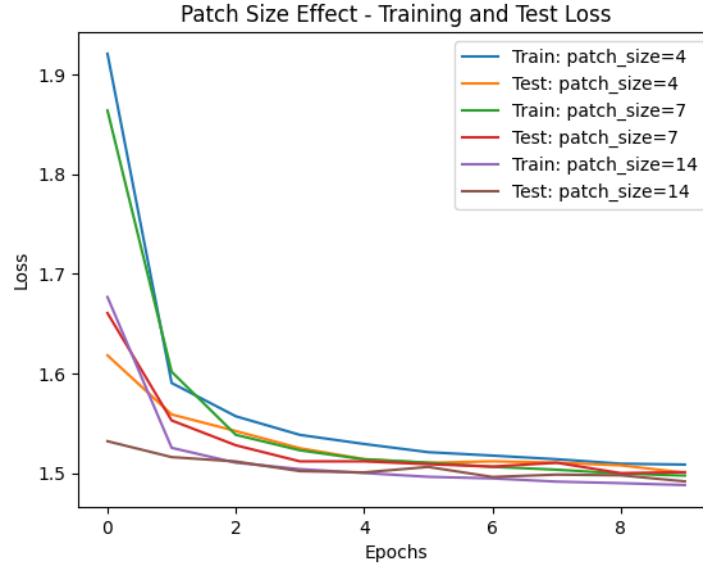


Figure 3.3: Effects of Patch Size

Patch size defines the dimensions of each segment when the image is divided. From the plot Figure 3.3, smaller patch_size values (e.g., 4) tend to have higher initial losses, but over time, smaller patches show better convergence rates and lower final losses. This is because smaller patches capture finer image details. However, very small patch sizes can lead to additional computational overhead and may capture too many details, which could impact generalization.

3.5.4 Experimenting nb_blocks

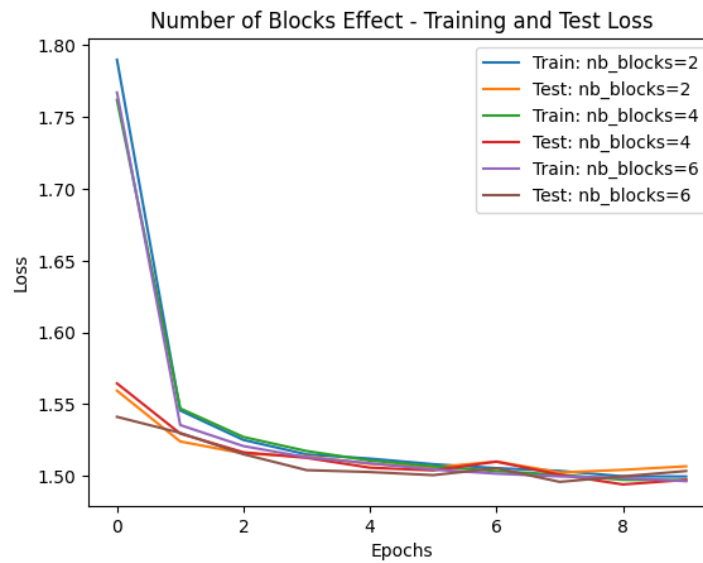


Figure 3.4: Effects of Number Of Blocks

The number of blocks refers to the number of stacked Transformer encoder layers (i.e., the layers of self-attention). From the plot Figure 3.4, we see that increasing `nb_blocks` (from 2 to 6) generally reduces both training and test losses, indicating that a deeper model can capture richer features. However, a higher `nb_blocks` count increases training time and memory requirements and may introduce overfitting risks.

Comment and discuss the final performance that you get. How to improve it?

In summary, the ViT model shows a consistent decrease in both training and test loss across different settings of embedding dimension, patch size, and number of blocks, with accuracy remaining around 96-97%, indicating that the model already performs well. To further improve performance, we can try more precise hyperparameter tuning (such as balancing embedding dimension and patch size) and incorporate data augmentation techniques (such as random cropping and color jittering) to enhance generalization ability. Additionally, due to limitations in computing power and model complexity, we trained the model for only 10 epochs, which is likely insufficient for the model to fully converge. Therefore, increasing the number of training epochs could further improve performance.

What is the complexity of the transformer in terms of number of tokens? How you can improve it?

The complexity of the transformer primarily lies in the computational complexity of the self-attention mechanism, which grows quadratically with the number of input tokens N , with a complexity of approximately $O(dN^2)$ (where d is the embedding dimension). This high complexity arises because, in the self-attention mechanism, each token needs to interact with all other tokens, requiring the computation of a full $N \times N$ attention matrix, with each token having an embedding dimension of d . To reduce this complexity, we can try sparse attention mechanisms, where each token only attends to a subset of other tokens instead of all, thereby reducing the computational load. Alternatively, linear attention mechanisms can lower the complexity to $O(dN)$ by avoiding the computation of the full attention matrix.

3.6 Larger transformers

What it is the problem and why we have it? Explain if we have also such problem with CNNs. As ViT takes RGB images, the input tensor should have 3 channels.

The ViT model, specifically `vit_base_patch16_224`, requires input images to be 224×224 pixels and have 3 RGB channels, as it is trained on ImageNet. For instance, MNIST images (28×28 pixels) need to be resized to 224×224 to avoid shape errors when using the `timm` library without pre-trained weights. This resizing can distort the images and affect performance.

In contrast, CNNs generally do not face this resizing issue until the fully connected layer,

as they can handle varying input sizes in earlier layers. However, CNNs still require the input tensor to have the correct number of channels (e.g., 3 for RGB), similar to ViT, which means that grayscale images like those from MNIST must be converted to have 3 channels.

There is a trick in timm that allows to use pretrained models with different image resolution and number of classes. Try to reload the model to be able to use on MNIST images:

The trick is to customize the model's input configuration based on the image size and number of channels. The `vit_small_patch16_224` model can be adapted to process smaller images, such as 28×28 , by specifying the appropriate image size during its creation. In contrast, the `vit_base_patch16_224` model expects larger images of 224×224 , which leads to an error when smaller images are used. Using pretrained weights allows the model to leverage knowledge gained from similar input sizes, enhancing its ability to learn and predict effectively.

redo the training with the pretrained ViT-S

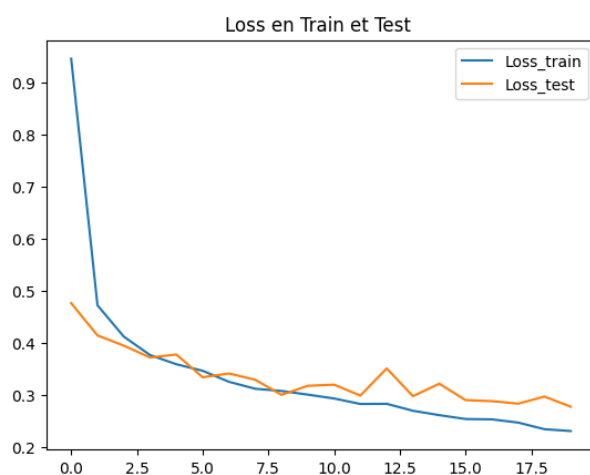


Figure 3.5: Training and testing loss curves for the pretrained ViT-S

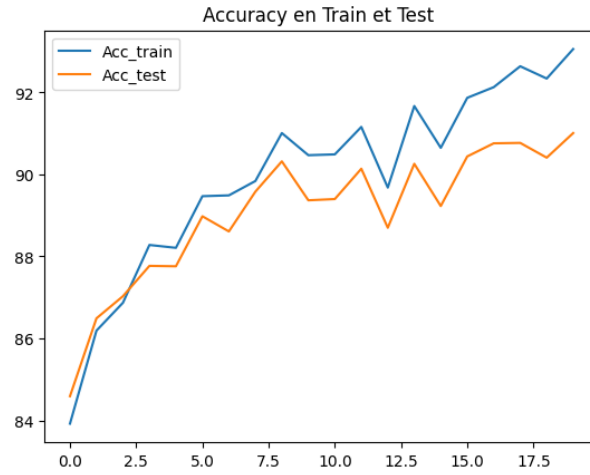


Figure 3.6: Training and testing accuracy curves for the pretrained ViT-S

redo the training but with the ViT-S pretrained on ImageNet

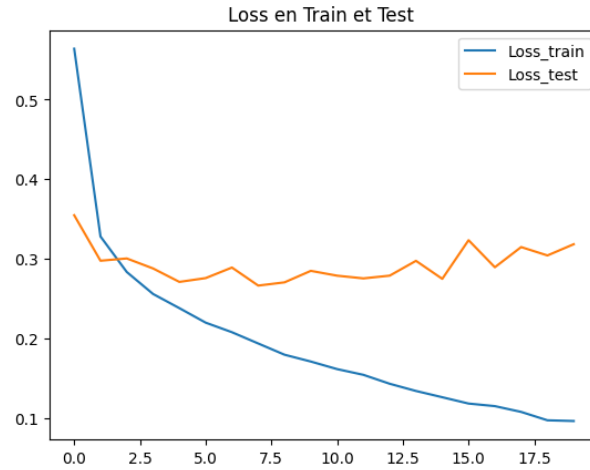


Figure 3.7: Training and testing loss curves for the pretrained on ImageNet

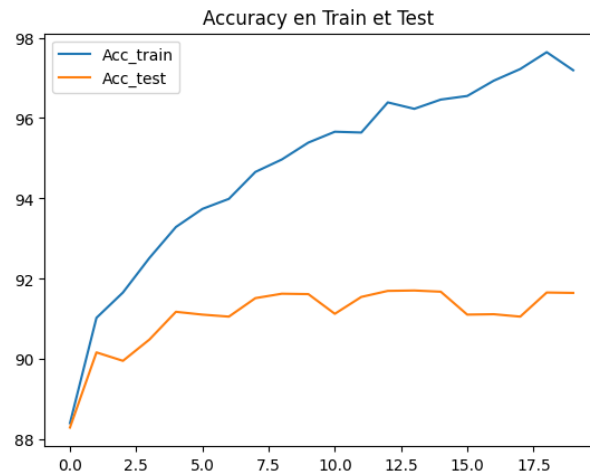


Figure 3.8: Training and testing accuracy curves for the pretrained on ImageNet

- The first model is **ViT-Base** (`vit_base_patch16_224`), which is larger, with more parameters and capacity.
- The second model is **ViT-Small** (`vit_small_patch16_224`), a smaller architecture designed to be more lightweight.

Pretrained Dataset:

- The first model was pretrained on a different dataset or from scratch for this experiment, which provides fewer generalization benefits.
- The second model was **pretrained on ImageNet**, a large-scale image dataset that provides a rich set of visual features, allowing the model to generalize better on new data.

The **second model (ViT-S pretrained on ImageNet)** performs better in both accuracy and loss:

- **Faster Convergence:** The pretrained ImageNet weights help the model converge more quickly, as shown by a steeper initial drop in loss.
- **Higher Accuracy:** It consistently achieves higher accuracy throughout training, due to richer feature representations from ImageNet.
- **Greater Stability:** The second model has smoother validation curves, indicating reduced overfitting and better generalization.

The **second model (ViT-S pretrained on ImageNet)** is smaller but leverages powerful pretrained features from ImageNet, making it more effective at generalizing, especially on smaller datasets.

Comment the final results and provide some ideas on how to make transformer work on small datasets. You can take inspiration from some recent work.

Fine-tuning a ViT on small datasets isn't easy, but there are smart ways to make it work. Using a smaller, customized model can make a big difference—reducing embedding dimensions and adjusting the number of heads per block helps the model focus on essential features without getting too complex. Pretrained models are great for this, as they come with built-in knowledge that can be adapted to small datasets, and adding data augmentation further expands our sample variety.

For ViTs to better capture details in smaller images, techniques like DINO and MAE can boost learning from limited labels, while methods like Locality Self-Attention (LSA) help the model pick up local patterns effectively. And finally, regularization methods like Dropout and Stochastic Depth are handy for preventing overfitting, which is key when working with limited data.

This combined approach helps ViTs perform much better in small-data settings.